Alexandria University

Faculty of Engineering

Computer and Communication Engineering Department

Computer Vision

Assignment 1

Fine Detail and the Big Picture

Technical report

Team members

#	Name	ID
1	Ezz Eldien Khaled Keleg	8806
2	Zeyad Essam Haridy	8936
3	Abdelrahman Ashraf Saad	8761

Overview and summary of results

This report walks through every part of the assignment in order. For each section it states what we implemented, shows every requested image with a caption that says what the image shows, and answers the questions asked in the assignment. Every captioned result carries an outcome tag: **SUCCESS**, **PARTIAL** or **FAILURE**, followed by the reason, so that the successes and the failures (which the assignment asks us to find and explain) are easy to locate.

Section	What we did	Headline result
1.1 Convolution	Loop, vectorized and SVD-based separable 2D convolution, written from scratch	All three match scipy to 3.6e-14 on 300 random cases; the vectorized version is ≈ 700× faster than the loop version at every size, and the separable version is up to 13× faster than vectorized (at 31×31)
1.2 Finite differences	Derivatives, gradient magnitude and edges on the façade image	Threshold 0.25 (8.6× the noise level); the frame lines are a zero-padding artifact (95.7 % vs 1.8 % of border pixels flagged)
1.3 DoG	σ = 2 derivative-of-Gaussian filters, single convolution	Threshold 0.08; noise level halved (0.029 → 0.013), smoother and connected edges
2.1 Hybrid images	Three hybrids with Fourier analysis, σ sweep and alignment test	Man + cat σ = 6 / 4 works; two portraits σ = 8 / 3 works with flaws; car + eyes σ = 6 / 3 fails (explained)
2.2 Aliasing	rings.png shrunk 4× with and without blur	Smallest σ with no false circles: 2.5 (false-pattern strength 0.999 → 0.028)
2.3 Pyramids	Own `reduce`/`expand`, Gaussian and Laplacian pyramids on a 670×736 image	Exact rebuild (error 1.7e-16) versus 0.267 for the "level minus blur" variant; total size 4/3 of the image
2.4 Blending	Oraple plus two own blends, level by level	Oraple (6 levels) and face-on-watermelon (5 levels) succeed; kite (3 levels) partial
3.1 Seams	Energy, dynamic program, removal of 30 % of the width	214 seams in 27.8 s; brute force would need 10^148 – 10^233 candidates
3.2 Comparisons	Seven images, resize vs crop vs seam carving with two energies	Seam carving wins on the tower, rocket, home and (with DoG) the person; neutral on the sunset; fails on Starry Night and on our building façade
Bonus	Object removal (bottle) and enlargement by seams	k = 125 seams; bottle removed, width restored; the "k seams from a copy" method stretches 125 columns versus 3.6 for repeated insertion

Conventions used in the report

- Pixel values are in [0, 1]. Derivatives, DoG filters, high-pass images and Laplacian levels go negative and are shown with 0 as mid-gray (clipping to [0, 1] only happens at display time).
- Part 1 pads with zeros, as requested. From Part 2 on, every blur and derivative uses reflection padding and our own separable convolution.
- Library functions are used only to load, save and show images, to convert color, to align images with `align_images.py`, to draw masks, to compute Fourier transforms, to resize the image in 3.2, and to check results (scipy in 1.1, the ideal filter in 2.2).
- All code is in the `code/` folder of the repository, all figures in `results/`, and the run logs in `results/logs/`. The excerpts of the logs that matter are reproduced in the appendices.

Part 1 — Filters and Edges

This part builds the one tool every other part depends on, a 2D convolution written from scratch, and then uses it to find edges with finite differences (1.2) and with derivative-of-Gaussian filters (1.3). Following the assignment, Part 1 pads with zeros; from Part 2 on, every blur uses reflection padding.

1.1 Convolution from scratch

Conventions. Images are 2D float64 arrays (colour images are processed channel by channel). Filters have odd height and width, so they have a centre pixel. Convolution is implemented as it is defined: the filter is flipped along both axes (`kernel[::-1, ::-1]`) and then correlated with the image. The input is padded with zeros by `(kh//2, kw//2)` so that the output has the same size as the input.

Three versions, all in `conv.py`:

- **Loop version** `conv2d_loop` — four nested loops: over every output pixel and over every filter entry.
- **Vectorized version** `conv2d_vectorized` — no loop over pixels. Each filter entry `(u, v)` contributes `flipped[u, v] × (image shifted by (u, v))`, and the shifted image is simply the slice `padded[u:u+H, v:v+W]`, so one whole-image NumPy operation is done per filter entry (kh·kw operations in total).
- **Separable version** `conv2d_separable` — computes the SVD of the kernel $K = U \cdot \mathrm{diag}(s) \cdot V^T$ and tests whether it has rank 1 up to rounding error (`s[1] ≤ 1e-10·s[0]`). If so, $K \approx \mathrm{col} \cdot \mathrm{row}$ (the weight is split evenly with √`s[0]`), and the image is convolved with the column and then with the row. Otherwise it falls back to the vectorized version.

Code snippet of the 1.1 convolution (complete, from `code/conv.py`):

```
import numpy as np

def _check_inputs(image, kernel):
    """Convert to float64 arrays and validate shapes."""
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    if image.ndim != 2:
        raise ValueError(f"image must be 2D (grayscale), got shape {image.shape}")
    if kernel.ndim != 2:
        raise ValueError(f"kernel must be 2D, got shape {kernel.shape}")
    kh, kw = kernel.shape
    if kh % 2 == 0 or kw % 2 == 0:
        raise ValueError(f"kernel height and width must be odd, got {kernel.shape}")
    return image, kernel



def pad_image(image, ph, pw, pad_mode="zeros"):
    """Pad `ph` rows on top/bottom and `pw` columns on left/right.

    pad_mode="zeros"   -> Part 1 (as the assignment asks)
    pad_mode="reflect" -> Parts 2 and 3 (np.pad(..., mode="reflect"))
    """
    widths = ((ph, ph), (pw, pw))
    if pad_mode == "zeros":
        return np.pad(image, widths, mode="constant", constant_values=0.0)
    if pad_mode == "reflect":
        return np.pad(image, widths, mode="reflect")
    raise ValueError(f"unknown pad_mode: {pad_mode!r}")



def conv2d_loop(image, kernel, pad_mode="zeros"):
    """2D convolution with explicit loops over every pixel and every filter entry.

    out[i, j] = sum_{u, v} flipped[u, v] * padded[i + u, j + v]
    where flipped = kernel flipped along both axes.
    """
    image, kernel = _check_inputs(image, kernel)
    H, W = image.shape
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2            # how far the filter reaches from its center
```

```
    padded = pad_image(image, ph, pw, pad_mode)   # shape (H + 2*ph, W + 2*pw)
    flipped = kernel[::-1, ::-1]                   # the "convolution" flip

    out = np.zeros((H, W), dtype=np.float64)
    for i in range(H):                  # loop over output pixels (rows)
        for j in range(W):              # loop over output pixels (columns)
            total = 0.0
            for u in range(kh):         # loop over filter entries (rows)
                for v in range(kw):     # loop over filter entries (columns)
                    total += flipped[u, v] * padded[i + u, j + v]
            out[i, j] = total
    return out



def conv2d_vectorized(image, kernel, pad_mode="zeros"):
    """2D convolution with NO loop over pixels: one whole-image operation per filter entry.

    Each filter entry (u, v) contributes  flipped[u, v] * (image shifted by (u, v)).
    The shifted image is just the slice padded[u:u+H, v:v+W] (a view, no copying).
    """
    image, kernel = _check_inputs(image, kernel)
    H, W = image.shape
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2

    padded = pad_image(image, ph, pw, pad_mode)
    flipped = kernel[::-1, ::-1]

    out = np.zeros((H, W), dtype=np.float64)
    for u in range(kh):                  # loop over filter entries only
        for v in range(kw):
            out += flipped[u, v] * padded[u:u + H, v:v + W]
    return out



def separable_factors(kernel, tol=1e-10):
    """SVD rank-1 test.  Returns (col, row) with kernel ~= col @ row, or None if not rank 1.

    K = U diag(s) Vt.  K has rank 1 (up to rounding) when the second singular value is
    negligible compared to the first:  s[1] <= tol * s[0].
    col has shape (kh, 1) (vertical filter), row has shape (1, kw) (horizontal filter).
    """
    kernel = np.asarray(kernel, dtype=np.float64)
    U, s, Vt = np.linalg.svd(kernel)
    if s[0] == 0.0:                                 # all-zero kernel: trivially separable
        return np.zeros((kernel.shape[0], 1)), np.zeros((1, kernel.shape[1]))
    if len(s) > 1 and s[1] > tol * s[0]:
        return None
    scale = np.sqrt(s[0])                           # split the weight evenly between the two
    col = (U[:, 0] * scale).reshape(-1, 1)
    row = (Vt[0, :] * scale).reshape(1, -1)
    return col, row



def conv2d_separable(image, kernel, pad_mode="zeros", tol=1e-10):
    """Convolution that exploits separability when the kernel has rank 1.

    rank 1     : convolve with the column, then with the row   (kh + kw multiply-adds / pixel)
    otherwise  : fall back to conv2d_vectorized                 (kh * kw multiply-adds / pixel)
    """
    image, kernel = _check_inputs(image, kernel)
    factors = separable_factors(kernel, tol)
    if factors is None:
        return conv2d_vectorized(image, kernel, pad_mode)
    col, row = factors
    tmp = conv2d_vectorized(image, col, pad_mode)     # vertical pass
    return conv2d_vectorized(tmp, row, pad_mode)      # horizontal pass
```

Sanity check against scipy

We ran 300 random test cases: small random images with random filters of odd heights and widths, of which 157 kernels were separable (outer products) and 143 were not. Every version was compared with `scipy.signal.convolve2d(..., mode="same")`:

Version	Max |ours – scipy| over 300 cases	Verdict
Loop	3.55e-14	matches to rounding error

Version	Max |ours – scipy| over 300 cases	Verdict
Vectorized	3.55e-14	matches to rounding error
Separable	2.13e-14	matches to rounding error

Timing on a 256×256 image

Gaussian filters from 3×3 to 31×31 were applied to a 256×256 image. Vectorized and separable times are the best of several repeats (as recorded in the run log).

Filter	Loop (s)	Vectorized (s)	Separable (s)	Loop ÷ vectorized	Vectorized ÷ separable
3×3	0.7407	0.00103	0.00255	719	0.4
5×5	1.8229	0.00236	0.00211	774	1.1
7×7	3.4207	0.00553	0.00322	619	1.7
9×9	6.0423	0.00825	0.00440	732	1.9
11×11	8.3810	0.00995	0.00324	842	3.1
13×13	11.1958	0.01732	0.00333	646	5.2
15×15	16.0649	0.01936	0.00377	830	5.1
17×17	19.9286	0.02566	0.00520	777	4.9
19×19	27.8159	0.03082	0.00472	903	6.5
21×21	31.3197	0.03849	0.00387	814	10.0
23×23	40.3912	0.05512	0.01379	733	4.0
25×25	45.9935	0.06299	0.00501	730	12.6
27×27	48.9960	0.06220	0.00740	788	8.4
29×29	58.3853	0.08721	0.00760	669	11.5
31×31	66.3780	0.09805	0.00753	677	13.0

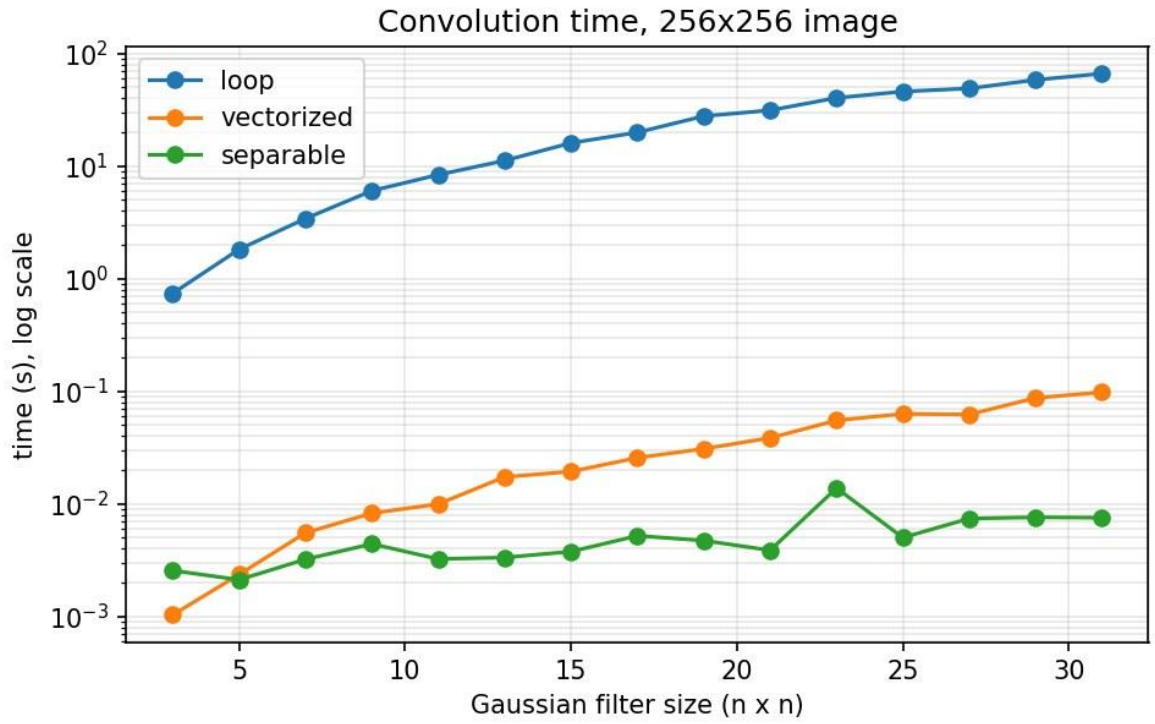


Figure 1. Convolution time versus Gaussian filter size (256×256 image, log scale). Blue: loop version. Orange: vectorized version. Green: separable version. The loop curve sits two to three orders of magnitude above the other two at every size; the separable curve is almost flat, drops below the vectorized curve from 5×5 on, and is 13× faster at 31×31.

SUCCESS: all three versions agree with scipy to rounding error, and the timings follow the expected cost model (explained below).

Why the times differ

- **Loop vs vectorized (≈ 620–900× at every size).** Both versions perform the same arithmetic, kh·kw multiply-adds per pixel. The loop version runs each one separately in the Python interpreter (65 536 pixels × k^2 iterations, each with indexing overhead), whereas the vectorized version issues only k^2 NumPy calls, each processing all 65 536 pixels in compiled C code. Both grow as k^2, which is why the ratio stays roughly constant. At 31×31 the loop takes 66.4 s and the vectorized version 0.098 s.
- **Vectorized vs separable.** Vectorized cost grows with k^2 array operations; the separable version needs only k (column) + k (row) = 2k after one tiny SVD, so the ideal gain is $k^2/2k = k/2$ (15.5 at 31×31). We measured 13.0×. For a 3×3 filter the separable version is actually slower (2.55 ms vs 1.03 ms, ratio 0.4) because the SVD and the two passes (each with its own padding and temporary array) cost more than the 9 → 6 operations they save. They break even at 5×5 and separability wins increasingly from there on.
- The separable version is therefore the one used for every blur and derivative from 1.2 onward (Gaussians and the box filter are separable). The single separable timing at 23×23 (13.8 ms) is an isolated outlier above the trend and looks like timing jitter on a machine that was not otherwise idle.

1.2 Finite difference operator

We filtered the grayscale façade image (670×736) with `D_x = [1 0 −1]` and `D_y`, its transpose, using our convolution (zero padding, as required in Part 1). The gradient magnitude is $\sqrt{(\partial I/\partial x)^2 + (\partial I/\partial y)^2}$, and the edge image is the magnitude thresholded at **t = 0.25** (pixel values in [0, 1]). Derivative maps are displayed with zero as mid-gray (scaled to ±0.514, the 99.5th percentile of the interior values); the arrays themselves are not modified.

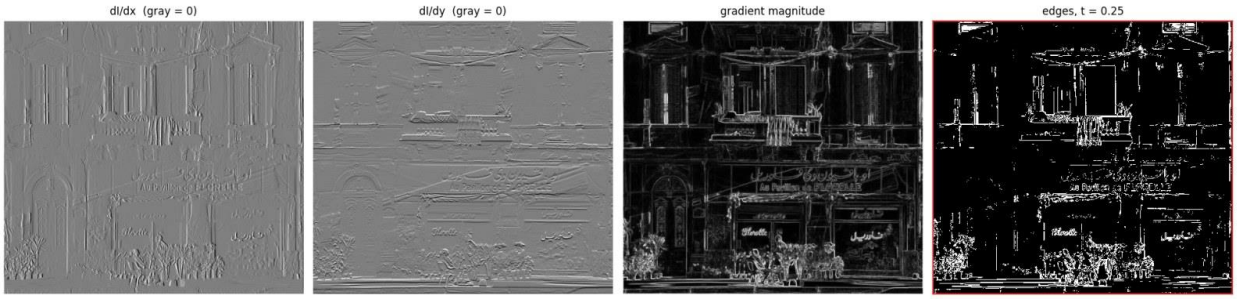


Figure 2. Finite differences on the façade: ∂I/∂x, ∂I/∂y, gradient magnitude and edge image (t = 0.25). ∂I/∂x responds to vertical structure (shutter slats, window sides, pillars) and ∂I/∂y to horizontal structure (balcony rail, cornices, the baseline of the shop sign). The magnitude combines both, and the threshold keeps the strongest 9.4 % of pixels. The red outline in the last panel marks the image boundary so that the white border lines are visible.

SUCCESS: the main edges (windows, balcony, lettering, cornices) are found, but the map is noisy: foliage and masonry texture produce many isolated fragments because a finite difference amplifies every pixel-level fluctuation. This is what 1.3 improves.

Justification of the threshold

- **It is well above the noise.** The robust noise level of ∂I/∂x (median absolute deviation) is 0.029, so t = 0.25 is 8.6× that level and isolated noise pixels do not pass. It is also 2.8× the mean interior gradient magnitude (0.090).
- **It keeps structure without clutter.** At t = 0.15, 18.0 % of the interior is flagged and plant leaves and stone texture turn into white clutter; at t = 0.35 only 4.3 % survives and real structure (letters of the sign, window frames, the overhead wire) starts to break up; at t = 0.45 (2.0 %) most of it is gone (see the threshold sweep, Figure 7, top row). 0.25 keeps 8.9 % of the interior, a good balance between the two.

Where do the lines along the image frame come from?

They are a **border artifact of zero padding**, not real edges. At the outermost pixels the filter window sticks out of the image, where we pretend the values are 0. For a bright pixel next to an imaginary black pixel the finite difference is large, so every pixel on the border looks like a sharp edge. Measured on the 1-pixel frame, the mean gradient magnitude is **0.478 with zero padding versus 0.041 with reflect padding** (the interior average is 0.090), and **95.7 % of the border pixels are flagged as edges with zero padding versus 1.8 % with reflect padding**. The artifact is stronger where the image is brighter (the correlation between border magnitude and brightness along the top row is 0.63), which is exactly what a step from the pixel value to 0 predicts. The line is 1 pixel wide because the filter reaches only 1 pixel. The remedy, used from Part 2 on, is reflect padding (or simply ignoring the frame).



Figure 3. The image frame in detail: top-left 110×110 corner of the façade, with edges computed using zero padding and using reflect padding. Left: the image corner. Middle: with zero padding a white line runs along the top and left borders. Right: with reflect padding the line is gone, while all genuine edges (the window grille at the top right) are identical in the two maps.

INFO: the frame lines are caused purely by the zero padding of the convolution.

1.3 Derivative of Gaussian (DoG) filter

We built a 2D Gaussian kernel with **σ = 2** and size 13×13 (the half-width is 3σ = 6 pixels), and filtered the kernel itself with `D_x` and with `D_y`; the two results are the DoG filters. Both have zero sum (−2.8e-16, so flat regions give no response), `DoG_y` is exactly the transpose of `DoG_x`, and the SVD test confirms they are rank 1 (separable). Each DoG filter is then applied to the image with one single convolution, and the steps of 1.2 are repeated with **threshold t = 0.08**.

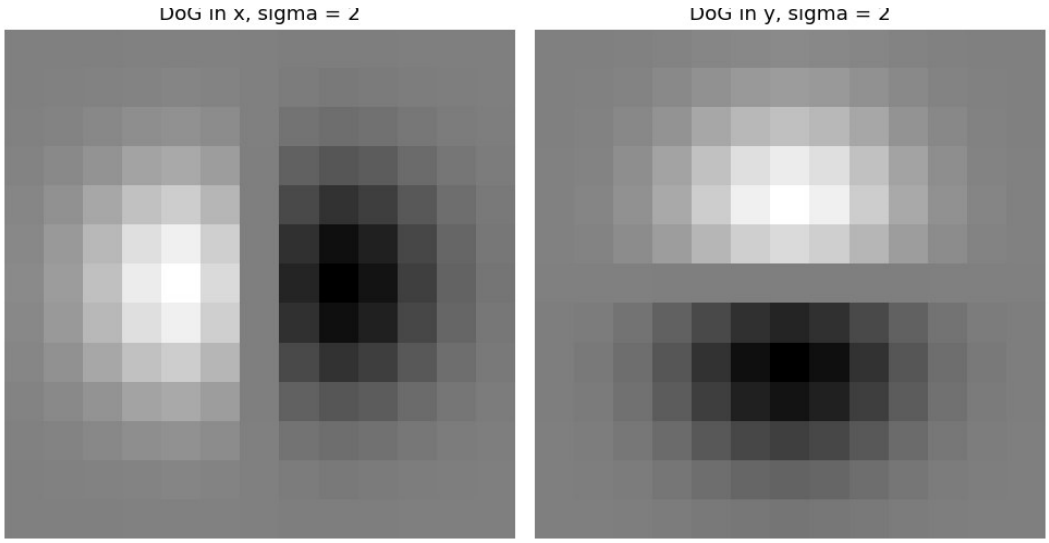


Figure 4. The two DoG filters, σ = 2, size 13×13 (gray = 0). Left: DoG in x, a positive lobe (white) beside a negative lobe (black): it responds to vertical edges and, because it is the derivative of a smooth bump, it also averages along the edge. Right: DoG in y, the same filter rotated by 90°.

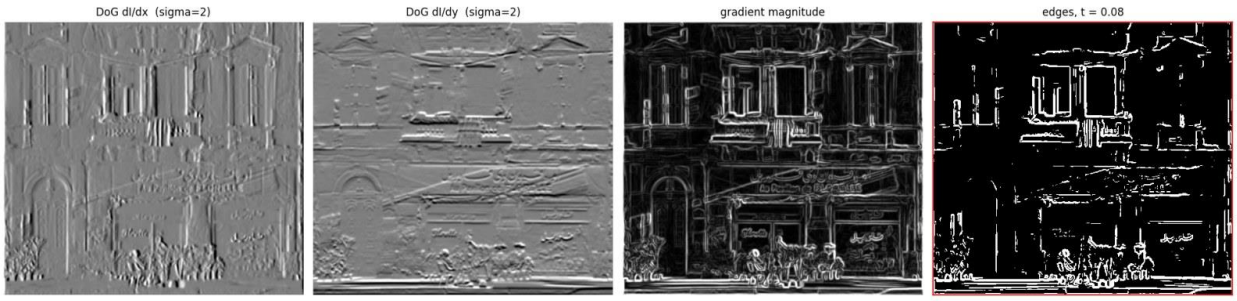


Figure 5. Derivative-of-Gaussian result: DoG ∂I/∂x, DoG ∂I/∂y, gradient magnitude and edge image (σ = 2, t = 0.08). Compared with Figure 2 the maps are smoother and the edge lines are continuous and thicker. The edge image closes the contours of the windows, shutters, balcony and arches that were broken into fragments in 1.2, and most foliage and wall-texture noise has disappeared.

SUCCESS: blurring before differentiating removed the noise while keeping the structural edges.

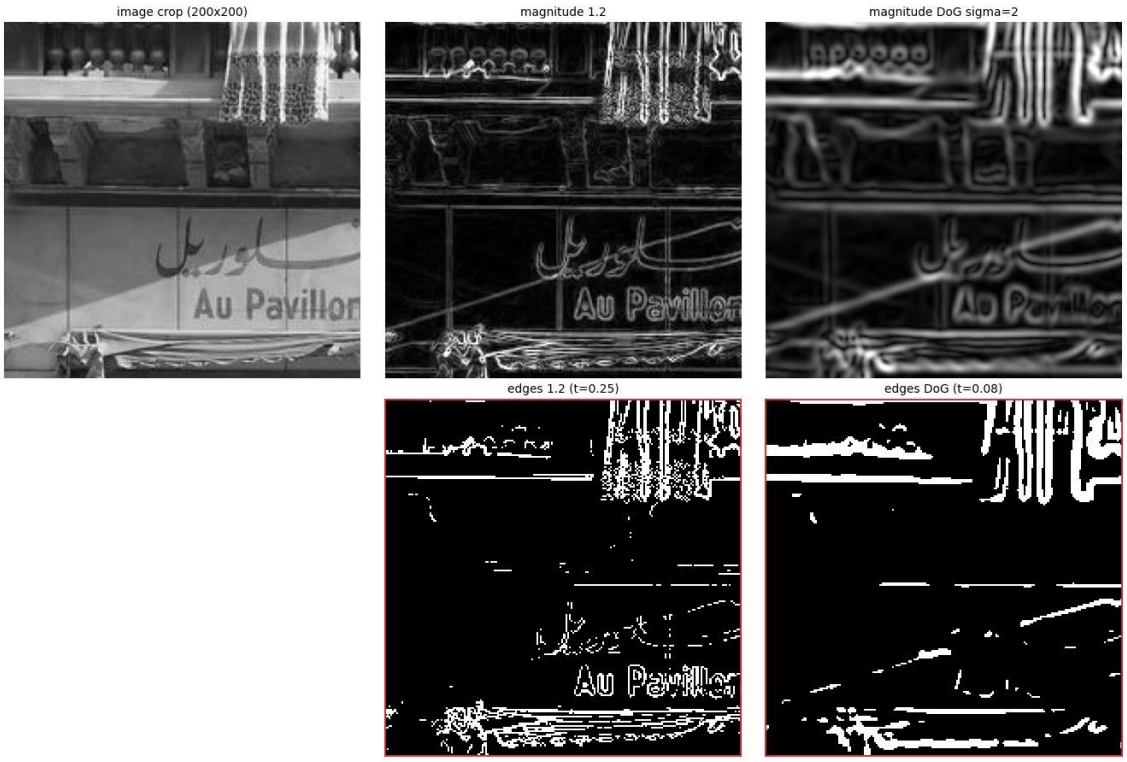


Figure 6. Close-up comparison of 1.2 and 1.3 on a 200×200 crop (sign "Au Pavillon"). Top row: crop, gradient magnitude of 1.2, gradient magnitude of the DoG. Bottom row: edges of 1.2 (t = 0.25) and of the DoG (t = 0.08). The finite-difference result outlines every letter and the texture of the curtain with thin 1-pixel lines; the DoG result gives wide, smooth bands that follow the large shapes (shadow line, ledges, curtain) but merge the small letters into blobs.

INFO: the DoG trades localisation and fine detail for noise robustness.



Figure 7. Threshold sweep for both methods. Top row: finite differences with t = 0.15, 0.25, 0.35, 0.45. Bottom row: DoG (σ = 2) with t = 0.05, 0.08, 0.11, 0.15. The percentage of interior pixels flagged is 18.0 / 8.7 / 4.3 / 2.0 % (top) and 22.6 / 10.2 / 4.6 / 1.5 % (bottom). The chosen values (second column) keep a comparable edge density, 8.7–8.9 % versus 10.2–10.3 %, so the two methods can be compared fairly.

Our σ and threshold, and how and why the maps differ from 1.2

Chosen values: σ = 2 (13×13 filter) and t = 0.08. σ = 2 is large enough to suppress pixel noise and texture, and small enough to keep the shapes of the shutters and balcony. The threshold is lower than in 1.2 because the DoG magnitude is smaller: its 99th percentile is 0.163 against 0.541, since the Gaussian averages an edge's contrast over a neighbourhood. t = 0.08 is 6.2× the DoG's noise level (0.013) and flags about the same share of the interior as 1.2 did (10.3 % vs 8.8 %).

Property	Finite differences (1.2)	DoG, σ = 2 (1.3)	Reason
Noise level of ∂I/∂x (MAD)	0.029	0.013	the Gaussian averages away pixel noise before the derivative
99th percentile of magnitude	0.541	0.163	edge contrast is spread over the Gaussian footprint
Edge appearance	thin, fragmented, noisy	thick, smooth, connected	an edge becomes a band about 2σ wide
Small detail (letters, leaves)	kept (and noise too)	merged or removed	details smaller than σ are blurred away
Localisation of the edge	≈ 1 px	a few px	the response peak is wide
Width of border artifact (zero padding)	1 px per side	6 px per side	the 13×13 filter reaches 6 px

In short: the maps differ because convolution is associative. Differentiating a Gaussian-smoothed image is the same as filtering once with the derivative of the Gaussian, so the DoG result is the 1.2 result of a blurred image. Derivatives amplify high frequencies, and noise lives there; the blur removes it first. The price is a loss of resolution (thicker edges, vanishing fine detail), controlled by σ.

Part 2 — Frequencies, Pyramids and Blending

A Gaussian blur keeps the low frequencies of an image, and the image minus its blur keeps the high ones. This part uses that split to mix two images (2.1), to understand what goes wrong when an image is shrunk (2.2), and to decompose and blend images at every scale (2.3, 2.4). As the assignment asks, from here on every blur pads by reflection (`np.pad(..., mode="reflect")`) and runs on the separable version of our own convolution.

2.1 Hybrid images

Pipeline for every pair. (1) Align the two images with `align_images.py` using two pairs of matching points (the eyes, or equivalent landmarks). (2) Low-pass the first image with a Gaussian of standard deviation σ_low. (3) High-pass the second image: the image minus its Gaussian blur with σ_high. (4) Add the two. The σ values set the cutoff frequencies: a Gaussian filter keeps half of a frequency's amplitude at $f_c \approx 0.1874 / \sigma$ cycles per pixel (a period of about 5.3σ pixels). The Fourier plots are the log magnitude of the shifted 2D FFT of the gray image, exactly as specified (`np.log(np.abs(np.fft.fftshift(np.fft.fft2(gray))))`). Colour versions are produced by filtering each channel separately.

Hybrid	Low-pass image (σ_low)	High-pass image (σ_high)	Alignment points (low ↔ high)	Outcome
1. Man + cat (starter pack)	man, σ = 6 (0.031 cyc/px, period 32 px)	cat, σ = 4 (0.047 cyc/px, period 21 px)	eyes: (299,341),(443,332) ↔ (605,290),(760,372)	SUCCESS
2. Own: two portraits	portrait with glasses, σ = 8 (0.023 cyc/px, period 43 px)	portrait without glasses, σ = 3 (0.062 cyc/px, period 16 px)	eyes: (310,452),(477,455) ↔ (316,401),(430,402)	SUCCESS (with flaws)
3. Own: car + eyes	car, σ = 6 (0.031 cyc/px, period 32 px)	close-up of eyes and glasses, σ = 3 (0.062 cyc/px, period 16 px)	headlights (115,158),(418,158) ↔ eyes (310,452),(477,455)	FAILURE (explained)



Figure 8. The three final hybrid images in colour (man + cat, portraits, car + eyes). Each hybrid is the colour result of the pipeline above, viewed at full size where the high-frequency image dominates.

Hybrid 1 — man + cat (our favourite): the whole process

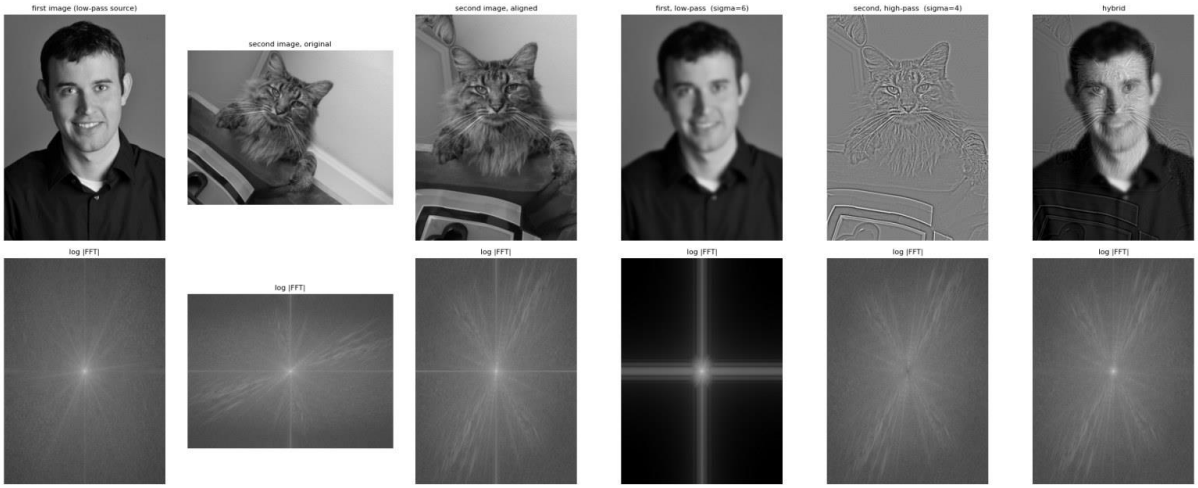


Figure 9. Complete process for man + cat, with the log-magnitude Fourier transform under every image. Top row, left to right: the man (low-pass source); the cat (original); the cat aligned to the man; the man low-passed (σ = 6); the cat high-passed (σ = 4); the hybrid. Bottom row: log |FFT| of each. Reading the spectra: the low-passed man has all its energy in a small bright central region and is black elsewhere (high frequencies removed). The high-passed cat is the opposite: a dark hole in the middle (low frequencies removed) with the energy kept at the periphery. The hybrid's spectrum contains both, the centre from the man and the outer region from the cat. The bright horizontal and vertical cross in the low-pass spectrum comes from the image borders: the FFT assumes the image repeats, and the jump between opposite borders produces energy along the axes (the cross is +3.81 in log-magnitude for the plain image and only +0.36 after multiplying by a Hann window).

SUCCESS: the two images separate cleanly in the frequency domain; the hybrid carries the man's low frequencies and the cat's high frequencies.



Figure 10. The man + cat hybrid at four viewing distances (full size, 1/2, 1/4, 1/8). Simulated viewing distance by shrinking the hybrid with our own `reduce`. At full size the fine details (fur, whiskers, cat eyes and ears) dominate and the face reads as a cat. At 1/4 and 1/8 size the fine details are no longer resolved and the face reads as the smiling man.

SUCCESS: the identity changes with viewing distance, as intended. σ_low = 6 and σ_high = 4 were chosen from the sweep below.



Figure 11. Why alignment matters: aligned versus misaligned by (80, 60) pixels. Left pair: 50/50 overlays of the two images. Right pair: the resulting hybrids. With correct alignment the cat's eyes, nose and mouth sit on the man's; when the cat is shifted the hybrid looks like a double exposure with two sets of eyes.

INFO: the two-point alignment (eyes to eyes) is essential for the illusion.

Tuning the two σ values

We swept σ_low ∈{4, 6, 8} and σ_high ∈{2, 3, 4, 5} and measured two things. Near (full size): the correlation of the hybrid's fine-detail band with the cat's (should be high) and with the man's (should be low). Far (shrunk to 1/8): the RMSE between the hybrid and the man (should be low) and the cat (should be high).

σ_low	σ_high	Near: corr. with cat	Near: corr. with man	Far: RMSE to man	Far: RMSE to cat
4	4	0.990	0.036	0.0116	0.2620
6	2	0.962	0.016	0.0141	0.2619
6	3	0.985	0.015	0.0150	0.2613
6	**4**	**0.993**	**0.015**	**0.0165**	**0.2606**
6	5	0.996	0.015	0.0186	0.2599
8	4	0.993	0.008	0.0223	0.2590

With σ_low = 6 and σ_high = 4, the cat's detail is almost entirely recovered up close (correlation 0.993) while the man leaks into it only 0.015, and from far away the hybrid is 16× closer to the man than to the cat. A smaller σ_low (4) lets mid-frequency man content overlap the cat's band (near-correlation with the man rises to 0.036); a larger σ_low (8) blurs the man so much that the far view loses his facial detail (RMSE 0.022). A larger σ_high lets more of the cat's mid-frequency content in, which starts to show from far away.



Figure 12. σ sweep, near view: full-resolution crop of the hybrid for σ_low ∈{3, 6, 9} (rows) and σ_high ∈{2, 4, 6} (columns). Increasing σ_high (to the right) brings in more of the cat (fur texture, whiskers); increasing σ_low (downwards) blurs the man more. With σ_high = 2 the cat is weak (top-left); with σ_high = 6 it is dominant.

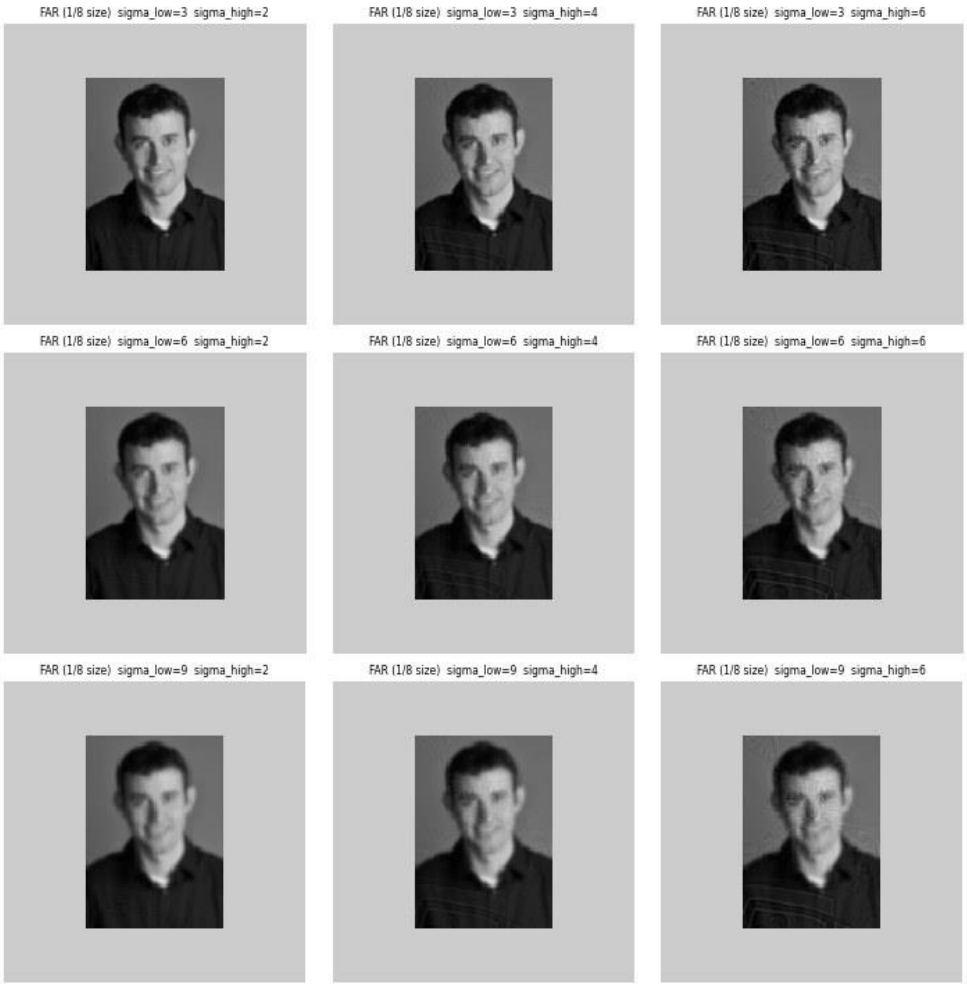


Figure 13. σ sweep, far view: the same nine hybrids shrunk to 1/8. From far away the nine hybrids all read as the man, but the smaller σ_low (top row) keeps his face sharper and the larger σ_high (right column) lets faint cat texture leak into the face.

Hybrid 2 — two portraits (own images)

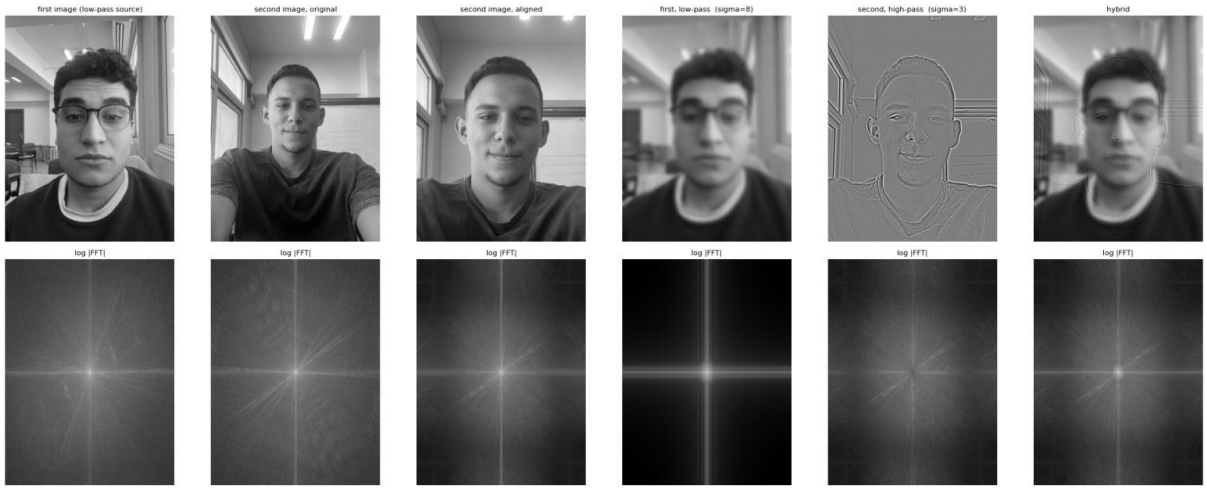


Figure 14. Complete process for the two portraits (σ_low = 8, σ_high = 3), with the Fourier transform under every image. The first portrait (glasses) is low-passed at σ = 8; the second portrait (no glasses) is aligned on the eyes and high-passed at σ = 3. The spectra behave as in Figure 9: the low-pass is concentrated at the centre, the high-pass has a dark centre, and the hybrid combines both.

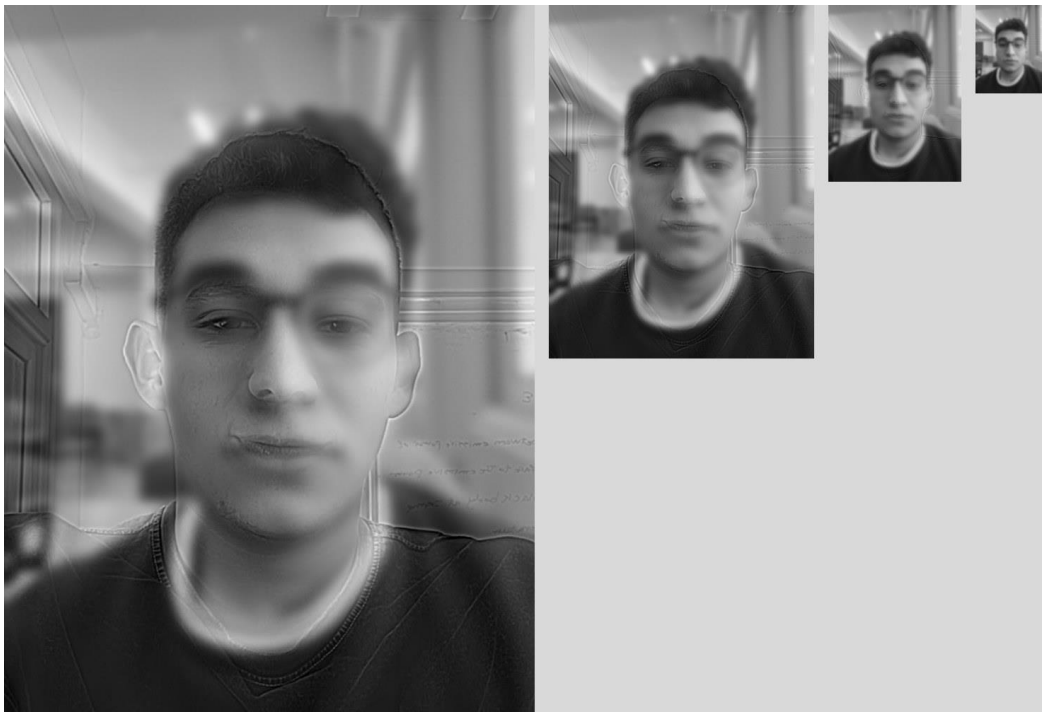


Figure 15. The portrait hybrid at four viewing distances. Up close the second face (no glasses) is read: the eyes, mouth, ears and the outline of the head come from the high-pass image. At 1/4 and 1/8 size the first face, with its glasses and dark sweater, takes over.

SUCCESS: the identity does change with distance. Two flaws remain. (1) The glasses of the low-pass face are thick dark bars, i.e. a low-frequency feature, so a dark band across the eyebrows stays visible even up close. (2) Because the backgrounds differ (a room with a window versus a different corner), the head outline and window edges of the high-pass image appear as faint contours around the head.



Figure 16. Alignment check for the portraits (aligned versus shifted by (80, 60) px). Aligned, the two faces coincide on the eyes and mouth; shifted, the hybrid shows two heads.

INFO: alignment on the eyes is what makes the pair work.

Hybrid 3 — car + eyes (own images): the one that fails

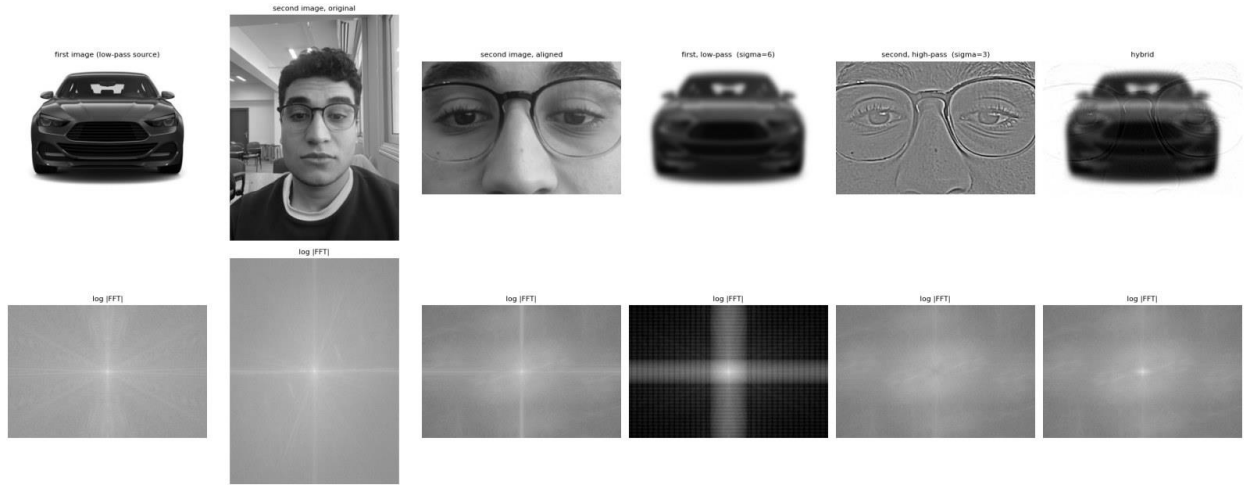


Figure 17. Complete process for car + eyes (σ_low = 6, σ_high = 3), with the Fourier transform under every image. The car (low-pass source) is blurred to a soft silhouette. The aligned second image is an extreme close-up of eyes and glasses, whose high-pass keeps only the glasses' rims, eyelashes and eyelids. In the hybrid the car is the dominant subject; the eyes and glasses appear as a faint overlay.

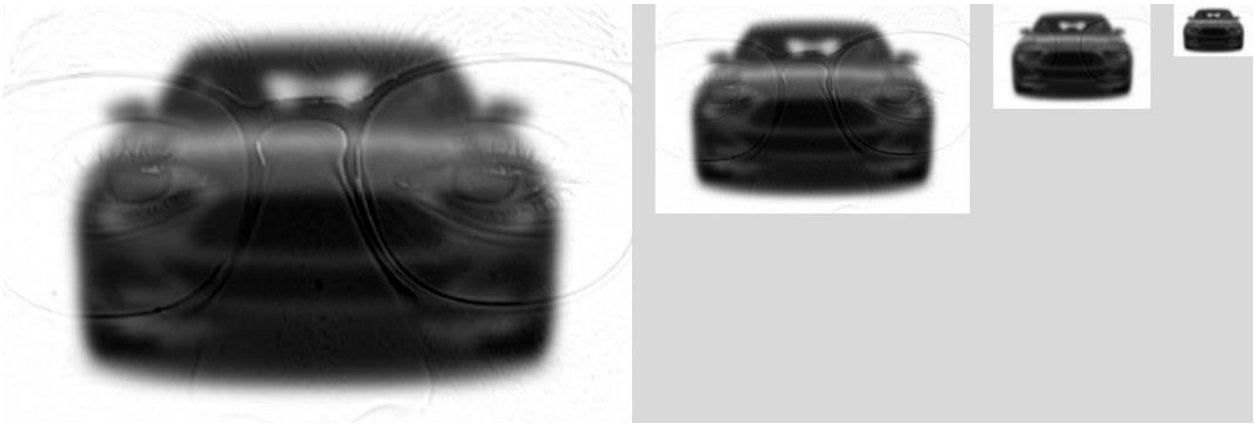


Figure 18. The car + eyes hybrid at four viewing distances. At full size we see the car with the glasses' outline and the eyes drawn over its windscreen and headlights; at 1/2 the glasses' rims are still visible; at 1/4 and 1/8 only the car remains.

FAILURE: the up-close view never reads as a face and the far view is not clean. Reasons: (1) **content mismatch**: aligning two point pairs (headlights ↔ eyes) forces a very large magnification of the face photo, so the high-frequency image contains only eyes and glasses, not a face-shaped gestalt that the viewer can recognise, while the car's large dark silhouette overwhelms it; (2) **dynamic range**: the car image has a pure white background, and adding the high-pass of a bright skin region pushes the hybrid to the range [−0.151, 1.453], so **17.49 % of the pixels are clipped** to white (against 1.47 % for man + cat), which erases contrast and leaves a white halo; (3) **frequency overlap**: the glasses' rims are thick, high-contrast lines, so even after high-passing at σ = 3 (period ≈ 16 px) they remain visible at half size and break the separation between the two distances.

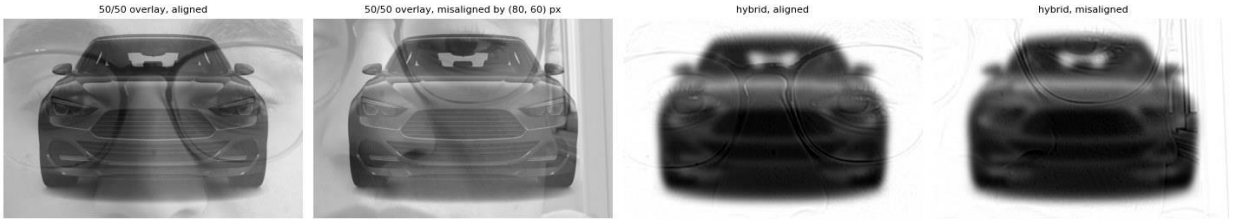


Figure 19. Alignment check for car + eyes. Even with the best two-point alignment the geometry of the two subjects does not correspond: only two points coincide, and the glasses' circles fall across the car's windscreen and bumper.

INFO: this is a pair that cannot be aligned well with a similarity transform (rotation, scale, translation), which supports the failure analysis above.

2.2 Aliasing

`rings.png` (512×512) is a zone plate: its local frequency grows with the distance from the centre (about 0.00085·r cycles per pixel). We shrank it to a quarter of its width and height (128×128) in two ways: (a) keeping every fourth pixel in each direction, (b) blurring with a Gaussian first and then keeping every fourth pixel.

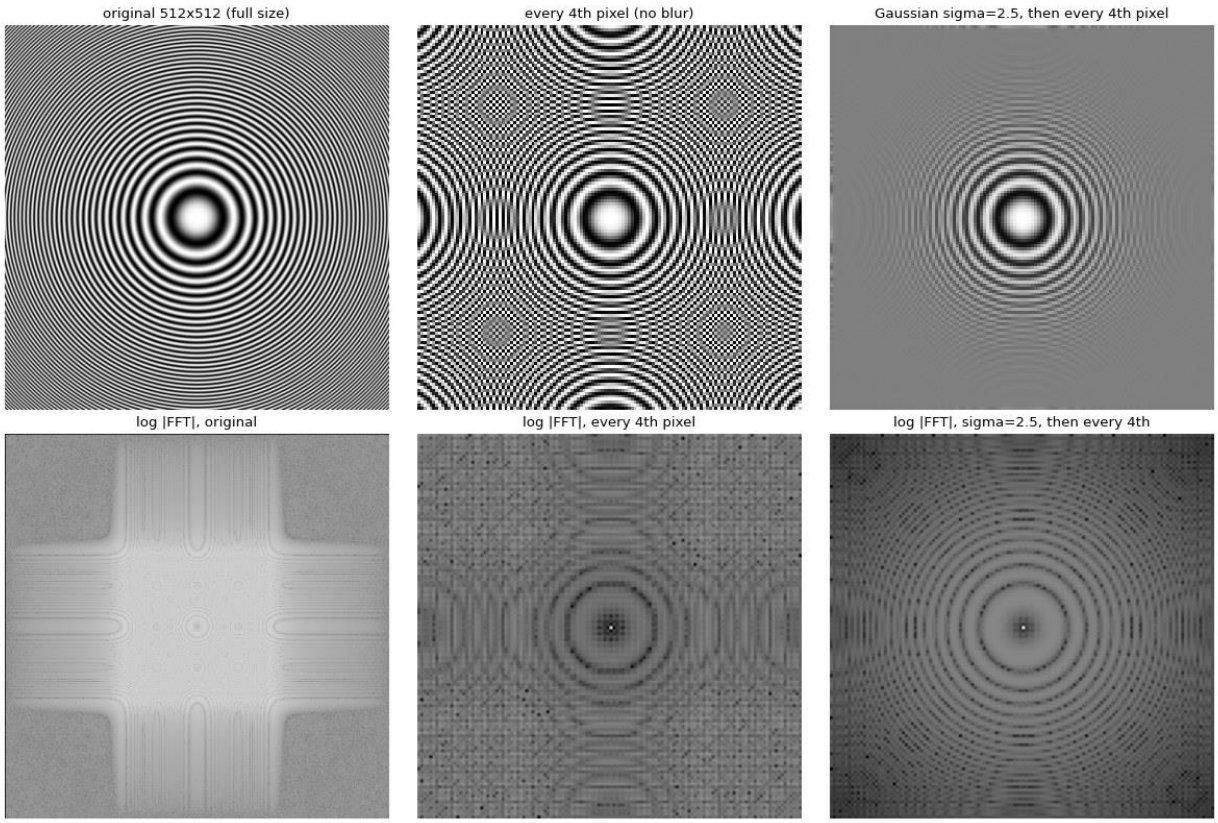


Figure 20. Aliasing experiment: original, every 4th pixel, and Gaussian σ = 2.5 then every 4th pixel, with the log Fourier magnitude of each. Top row: the original rings; the naïve 4× subsampling, full of false circles and checkerboard patterns away from the centre; and the blurred-then-subsampled result, with only the central rings and a flat gray outside. Bottom row: their spectra. The original spectrum fills the whole frequency square; the naïve result shows the spectrum folded back on itself (aliasing); the blurred result is clean and confined to the low frequencies.

SUCCESS: blurring before subsampling removes the false circles. (The original panel is itself drawn scaled down by the figure, so it may show a little screen aliasing; view `rings.png` at 100 % for the true image.)

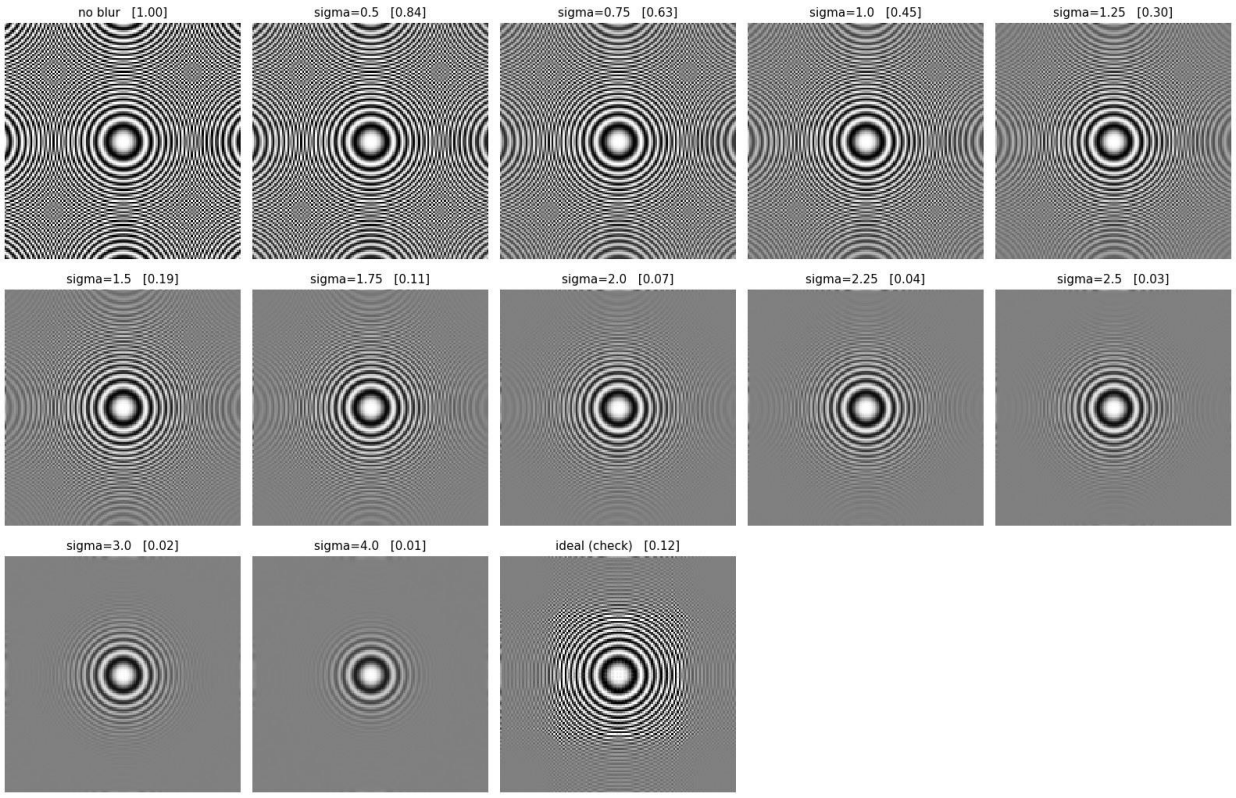


Figure 21. Sweep of the blur σ before taking every 4th pixel. The number in brackets is the strength of the false pattern. Strength is the RMS deviation from flat gray measured in the region where the correct shrunk image must be flat (1.00 = full-amplitude rings). The last panel is an ideal alias-free reference made in the Fourier domain, used only as a check.

σ	none	0.5	1.0	1.5	1.75	2.0	2.25	2.5	3.0	4.0
False-pattern strength	0.999	0.836	0.446	0.187	0.113	0.068	0.042	**0.028**	0.018	0.013
RMSE to ideal	0.287	0.245	0.148	0.106	0.109	0.118	0.130	**0.141**	0.159	0.180
Gain at the new Nyquist (1/8 cyc/px)	1	—	—	0.50	—	0.29	—	**0.145**	0.062	—

Smallest σ at which we no longer see false circles: σ = 2.5. At σ = 2.0 and 2.25 faint circular bands are still visible near the left and right edges of the result (strength 0.068 and 0.042). At σ = 2.5 the outside is flat gray and the strength is 0.028 (against 0.999 for plain subsampling). This is a judgement by eye: numerically, the strength falls below that of the ideal Fourier filter (0.119, which itself rings because of its sharp cutoff) already at σ ≈ 1.75, but faint circles are still perceptible there. Past σ = 2.5, extra blur only removes real detail (the RMSE to the ideal result grows from 0.106 at σ = 1.5 to 0.180 at σ = 4).

What happened in each result, and why

- **Keep every 4th pixel (no blur).** Sampling every fourth pixel means the new image can only represent frequencies below the new Nyquist limit, 1/(2·4) = 0.125 cycles per pixel. The rings image contains frequencies far above it (the local frequency passes 0.125 at a radius of about 147 px). Those frequencies are not discarded: they **fold back (alias) onto lower frequencies**, and appear as false circles, Moiré patterns and checkerboards (the folded spectrum in the bottom-middle panel). The false circles are the folded-back copy of the high-frequency rings.
- **Blur first, then keep every 4th pixel.** A Gaussian of standard deviation σ attenuates a frequency f by $\exp(-2\pi^2\sigma^2 f^2)$. At the new Nyquist limit f = 0.125 this is 0.50 for σ = 1.5, 0.29 for σ = 2.0 and 0.145 for σ = 2.5, and much less beyond. The frequencies that would have aliased are removed before sampling, so only the genuine central rings (frequencies below about 0.125) survive and the rest of the image is correctly flat gray.

Consequence for the Gaussian pyramid. A pyramid must never subsample a raw image: **every level must be low-pass filtered before every second row and column is dropped** (our `reduce` does blur first, then subsample). Building each level from the previous one also keeps each blur small, because a single step only needs to remove one octave of frequencies. The 5-tap kernel [1 4 6 4 1]/16 has response $\cos^4(\pi f)$, 0.25 at the new Nyquist limit of 0.25 cycles per pixel, so a small amount of aliasing leaks (for example 0.119 of a sine at f = 0.30). Reducing `rings.png` twice with our `reduce` leaves a false-pattern strength of only 0.031, comparable to a single σ = 2.5 blur-and-subsample.

2.3 Gaussian and Laplacian pyramids

We wrote two functions, both blurring with the separable kernel (1/16)[1 4 6 4 1] along x and along y (reflect padding):

- `reduce` blurs the image and then keeps every other row and column: (H, W) → (⌈H/2⌉, ⌈W/2⌉).
- `expand` places each pixel at the top-left of a 2×2 block of zeros, blurs, and multiplies by 4 (three pixels in four are zeros, so the brightness would otherwise drop to a quarter). When the level being rebuilt has an odd side, the expansion has one row or column too many and the last one is dropped.

```
def reduce(image):
    """Blur, then keep every other row and column.  (H, W) -> (ceil(H/2), ceil(W/2))."""
    image = np.asarray(image, dtype=np.float64)
    return _blur(image)[::2, ::2]


def expand(image, out_shape=None):
    """Put each pixel at the top-left of a 2x2 block of zeros, blur, multiply by 4.


    (h, w) -> (2h, 2w).  If `out_shape` is given it must be (2h, 2w) or (2h-1, 2w-1)-style: when the
    level we are rebuilding had an ODD side, the expansion has one row/column too many, and the last
    one is dropped.
    """
    image = np.asarray(image, dtype=np.float64)
    h, w = image.shape[:2]
    up = np.zeros((2 * h, 2 * w) + image.shape[2:], dtype=np.float64)
    up[::2, ::2] = image                                    # 3 pixels in 4 are zeros ...
    out = 4.0 * _blur(up)                                   # ... so multiply by 4 to keep the brightness
    if out_shape is not None:
        th, tw = out_shape[:2]
        if th not in (2 * h, 2 * h - 1) or tw not in (2 * w, 2 * w - 1):
            raise ValueError(f"cannot expand {(h, w)} to {(th, tw)}")
        out = out[:th, :tw]                                 # drop the extra last row / column
    return out
```

The Gaussian pyramid applies `reduce` repeatedly. The Laplacian pyramid stores, for each level, `L_i = G_i − expand(G_{i+1})` and ends with the smallest Gaussian level itself. To rebuild, we start from the smallest level and repeat: expand, then add the Laplacian level of that size. We used the colour façade image (670×736, sides that are not powers of 2) and six levels. No `cv2.pyrDown`, `cv2.pyrUp` or scikit-image pyramid function is used.

Level	Gaussian / Laplacian shape	Pixels per channel	Share of the image
0	670 × 736	493 120	1
1	335 × 368	123 280	0.250
2	168 × 184	30 912	0.063
3	84 × 92	7 728	0.016
4	42 × 46	1 932	0.004
5	21 × 23	483	0.001
Total		**657 455**	**1.33326** (4/3 = 1.33333)



Figure 22. Gaussian pyramid of the façade image (six levels, each half the size of the previous one). Level 0 is the original. Every level is a blurred, half-size copy of the one before; the odd sizes (335, 21, 23, …) are handled by rounding up and by dropping the extra row/column on expansion.

SUCCESS: the pyramid is built entirely by our own `reduce`, on an image whose sides are not powers of 2.



Figure 23. Laplacian pyramid of the same image, contrast boosted (gray = 0). Each level contains the detail lost between two consecutive Gaussian levels: the finest level holds edges and fine texture (shutters, lettering, ironwork), the coarser levels hold ever larger structures, and the last level (bottom right, tiny) is the smallest Gaussian level itself, shown in colour. Values are displayed as 0.5 + 1.99·L and clipped.

SUCCESS: the bands separate by scale as expected.



Figure 24. Rebuilding the image from the Laplacian pyramid, and the reconstruction error compared with the lecture's version. Left: the original. Second: the image rebuilt from our Laplacian pyramid, with maximum error 1.7e-16 (RMSE 2e-18; exactly zero in 99.6 % of the values). Third: the absolute error of our rebuild, drawn on a scale in which white = 0.160, is completely black. Right: the absolute error of the "each level minus its blur" variant on the same scale; its maximum error is 0.267 and its RMSE 0.049, and it follows the edges of the image.

SUCCESS: the rebuild is exact to floating-point rounding.

Why our rebuild is exact and the lecture's version is not

By definition `L_i = G_i − expand(G_{i+1})`, so `G_i = L_i + expand(G_{i+1})` is an identity, **as long as the decoder uses the very same `expand` as the encoder**. Starting from the stored smallest level and applying that identity level after level returns every `G_i` exactly, up to floating-point rounding (the odd-size crop is applied identically in both directions). It does not matter how good or bad expand is as an interpolator: whatever it fails to predict is stored in `L_i`. In the lecture's variant each level is stored as `G_i − blur(G_i)`; that difference is measured against a quantity the decoder never has (the decoder only has the coarser level, which is a sub-sampled copy of the blur). The difference between the decoder's prediction and `blur(G_i)` is not stored anywhere, so it reappears as a reconstruction error (maximum 0.267, RMSE 0.049 here, concentrated on edges).

Total size of the Gaussian pyramid

Each level has a quarter of the pixels of the previous one (half the rows times half the columns), so the total is the geometric series 1 + 1/4 + 1/16 + … = 1/(1 − 1/4) = **4/3 of the image (about 33 % extra)**. We measured 657 455 / 493 120 = 1.33326: slightly below 4/3 because the series is cut after six levels, and slightly above it because odd sides are rounded up. The Laplacian pyramid has exactly the same level sizes, hence the same total; it is not smaller than the image in number of samples, but most of its values are close to zero, so it is much more compressible.

2.4 Multiresolution blending

Blending follows the assignment: build the Laplacian pyramids `L1`, `L2` of the two images and the Gaussian pyramid of the mask `M`; at each level combine `M·L1 + (1 − M)·L2`; rebuild from the combined pyramid as in 2.3. Masks are kept sharp (white = 1 where the first image is wanted). Each blend is shown with its two inputs, its mask, the hard cut and the multiresolution result.

Blend	Images	How the mask was made	Levels
1. Oraple	apple.jpg + orange.jpg (512×512)	Left half white, right half black (a step mask built directly in code, columns 0–255 = 1).	6
2. Own: face on a watermelon	portrait photo (object) on a watermelon cut-out, 600×600	Irregular, face-shaped polygon with 14 vertices drawn with `cv2.fillPoly` in the watermelon's coordinates. The portrait was placed with `align_images.py` using the eyes ((310,452),(477,455)) mapped to (245,265),(355,265).	5
3. Own: kite in the sky	kite photo (object) on a cloudy sky	Two polygons drawn with `cv2.fillPoly` on the kite image (the kite's body and its ribbon tail), then warped with the same similarity transform as the image. Alignment points: the kite's top and bottom tips (328,89),(376,196) → (250,70),(298,177).	3

Blend 1 — the oraple

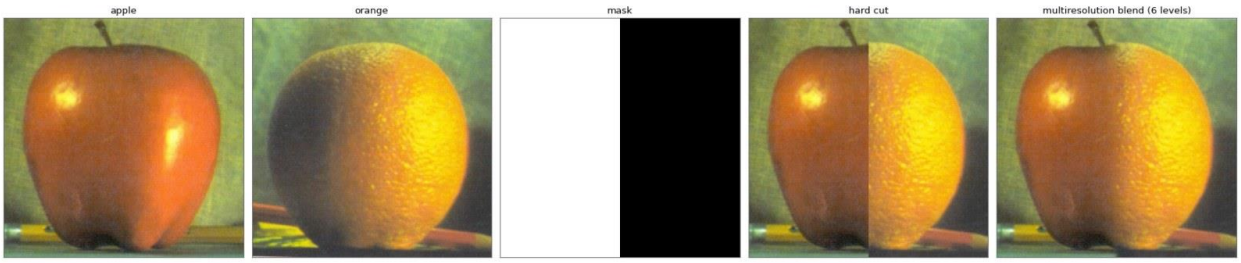


Figure 25. The oraple: apple, orange, mask, hard cut, and 6-level multiresolution blend. The hard cut shows an obvious vertical line where the red apple meets the yellow-orange orange, with a brightness and colour step. The multiresolution blend has no visible boundary: the skin changes gradually from apple to orange and the textures (the orange's pores, the apple's highlights) stay sharp on their own side.

SUCCESS: six levels make the transition about 67 px wide (13 % of the image) for the lowest frequencies, which hides the colour step, while the fine texture is switched over within a few pixels.

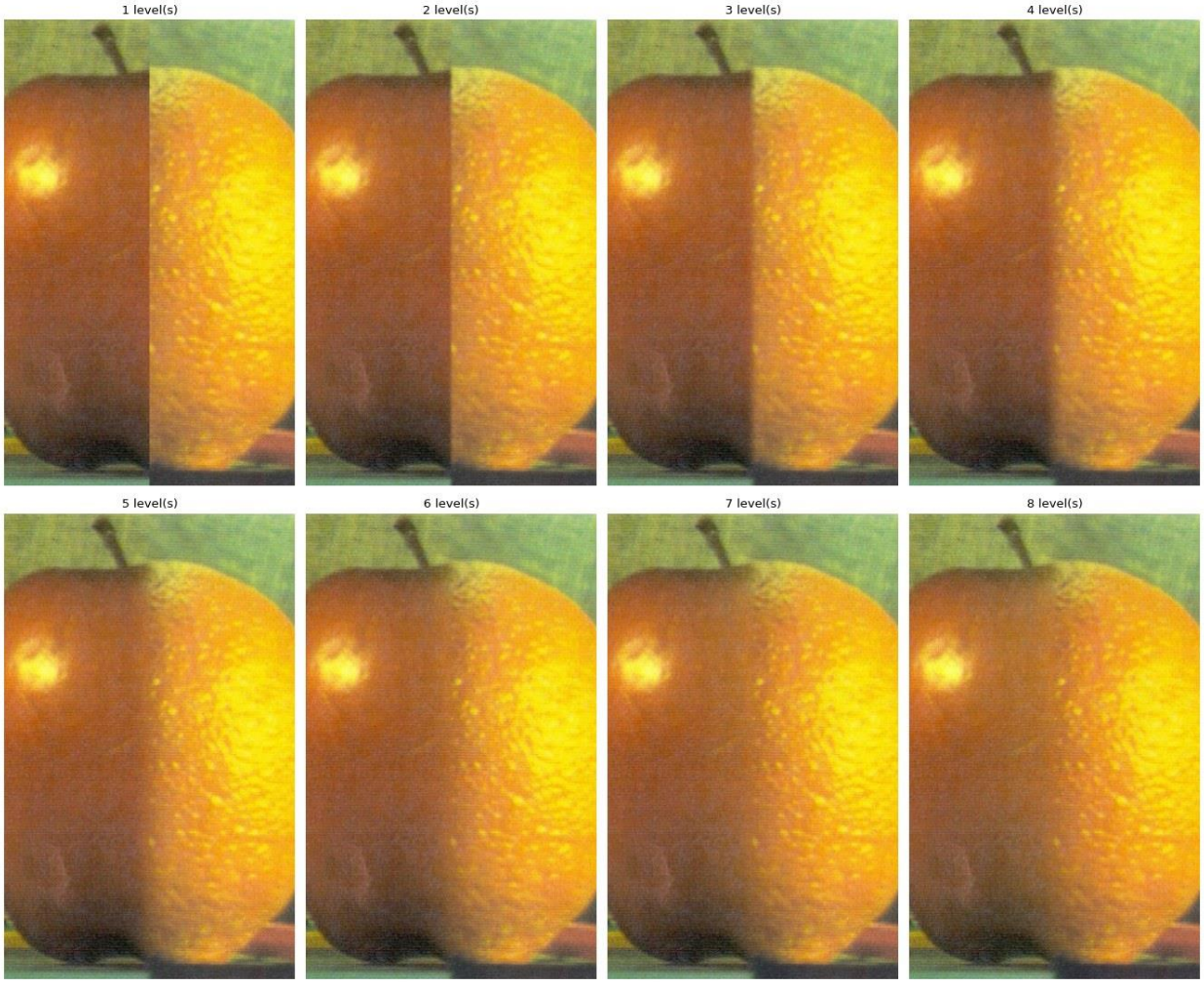


Figure 26. Choosing the number of levels for the oraple: 1 to 8 levels. With 1–3 levels the vertical seam is clearly visible (the transition is 0, 4 and 8 px wide). From 4–5 levels it is hardly noticeable, and at 6–8 levels it is invisible; at 8 levels the mixing zone reaches 273 px (more than half of the image), which makes the middle of the fruit a muddy average of both. The seam width (pixels where the first image's weight is between 10 % and 90 %) is 0, 4, 8, 16, 34, 67, 134, 273 px for 1 … 8 levels.

INFO: 6 levels is the smallest depth with no visible seam without over-mixing the two fruits.

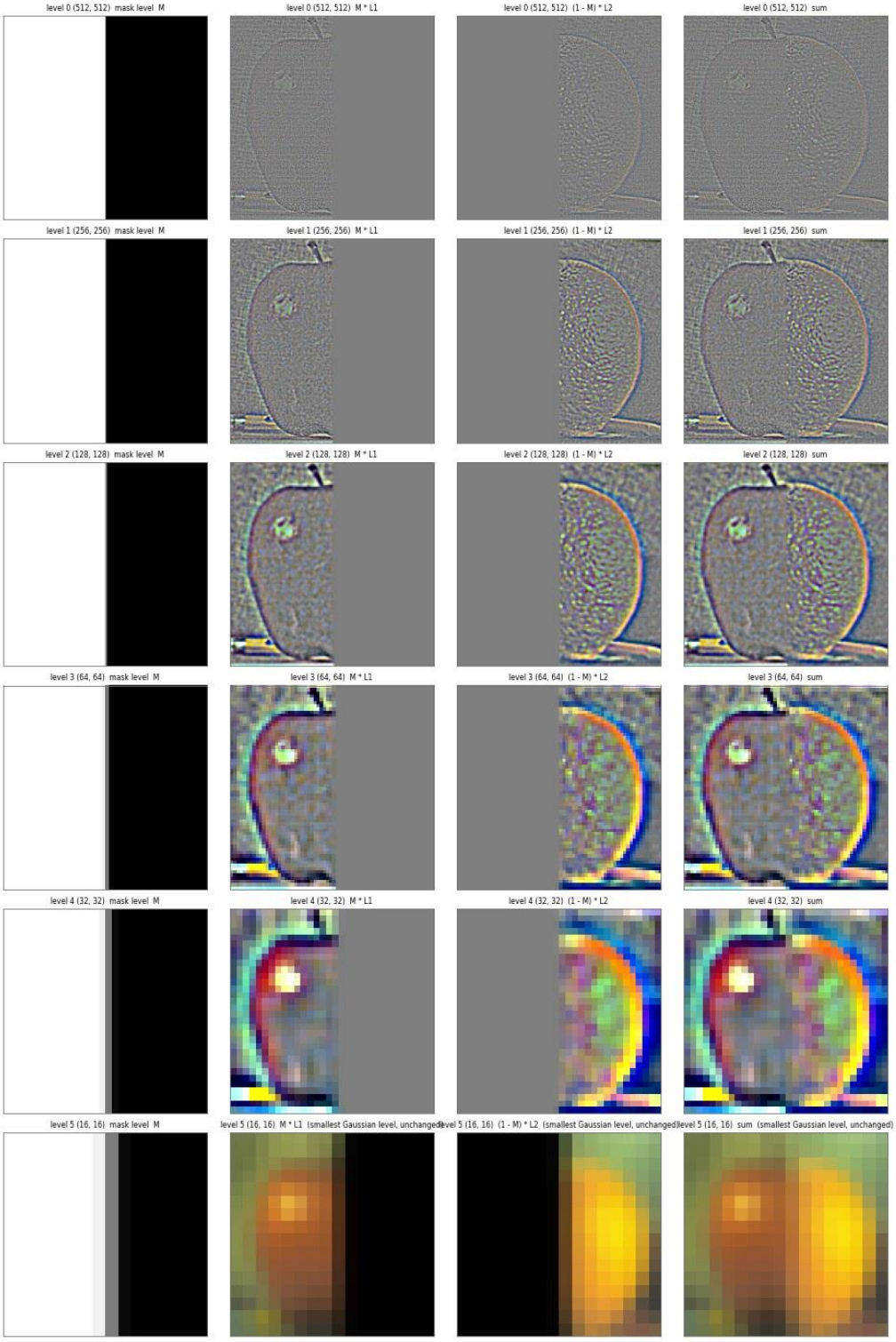


Figure 27. Level by level for the oraple: the mask M, M·L1, (1 – M)·L2 and their sum, for levels 0 (finest) to 5 (coarsest). The mask is a sharp step at level 0 and becomes a progressively wider gray ramp at coarser levels (the Gaussian pyramid of the mask). At fine levels the sum of the two halves switches abruptly, which is harmless because those levels contain only thin edges and texture; at the coarse levels the two parts fade into each other over a wide band, which is where the colour and brightness differences live. The last row is the smallest Gaussian level, which is blended directly.

INFO: each frequency band is blended over a width proportional to its scale.

Blend 2 — face on a watermelon (our favourite)

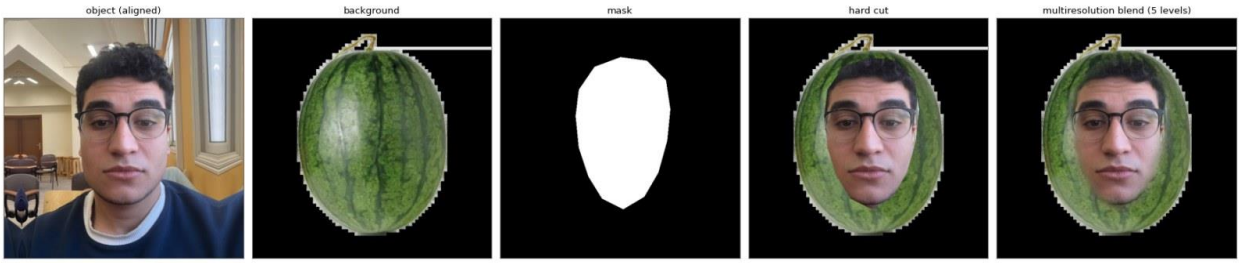


Figure 28. Face on a watermelon: object (aligned), background, mask, hard cut, 5-level blend. The hard cut pastes the face with a crisp polygonal outline and a clear mismatch of tone (pink skin and black hair against the green rind). In the blend the outline dissolves: the hair fades into the green stripes and the cheeks into the rind, while the eyes, glasses and mouth stay sharp because the fine bands are blended over only a few pixels.

SUCCESS: five levels give a transition wide enough for the colour difference but small enough that the face keeps its details. Remaining flaws: a soft greenish-pink fade zone around the face (the price of the wide low-frequency band) and the blocky, jagged edge of the original watermelon cut-out, which is unrelated to the blend. 3.4 % of the blended values fell slightly outside [0, 1] and were clipped for saving.

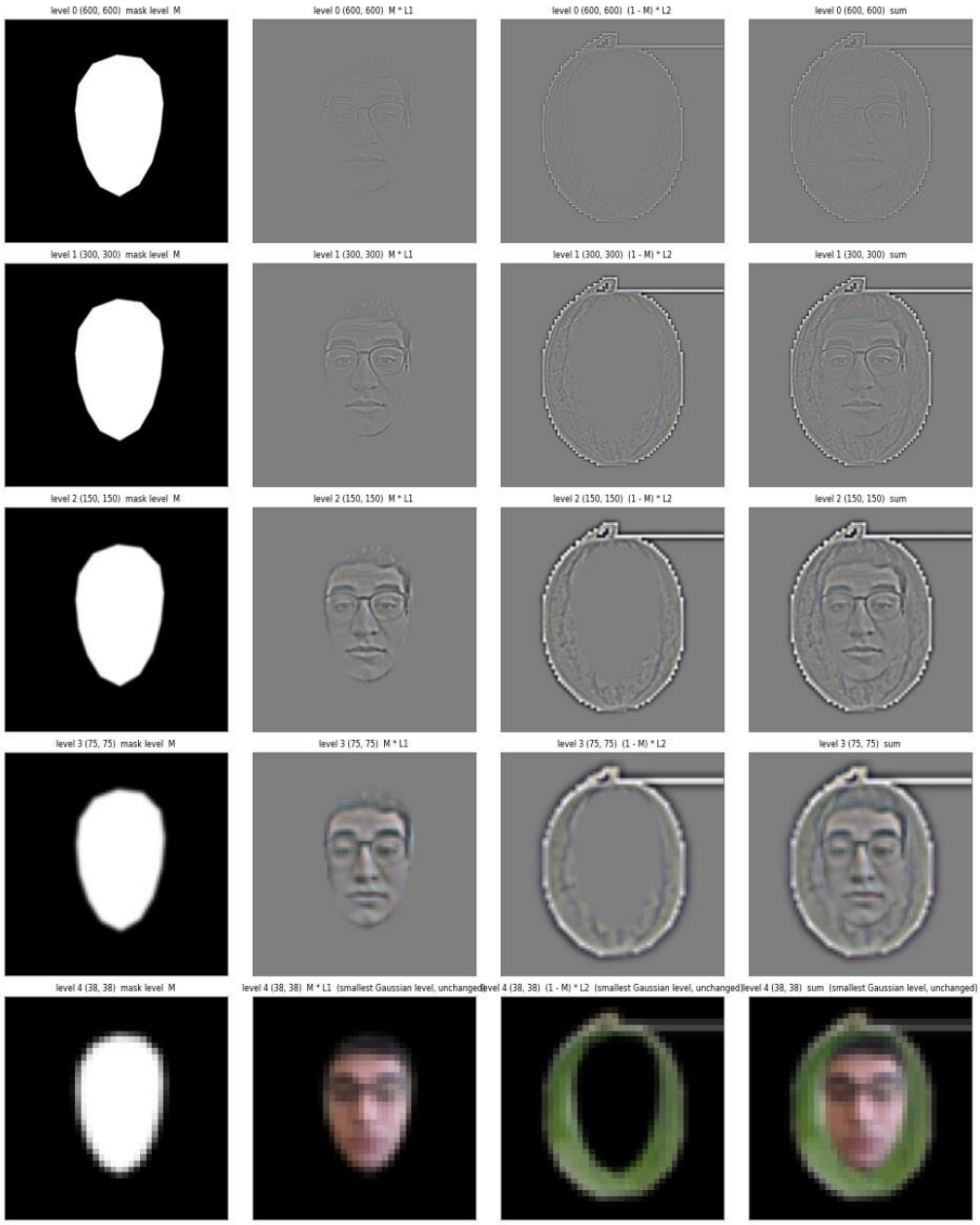


Figure 29. Level by level for the watermelon blend (levels 0 to 4), shown as mask M, M·L1, (1 – M)·L2 and the sum. Level 0 holds the thinnest edges (glasses, eyelids; the cut-out's edge). Moving to coarser levels the mask gets blurrier (wider transition) and the layers carry larger structures (face shape, rind stripes). The last row is the smallest Gaussian level: the blurred face on the left, the green ring of the melon on the right, and a soft boundary between them.

INFO: the sum of the two masked layers at every level is what the rebuild uses; the visible smoothness of the final face boundary comes from the coarse levels.

Blend 3 — kite in the sky

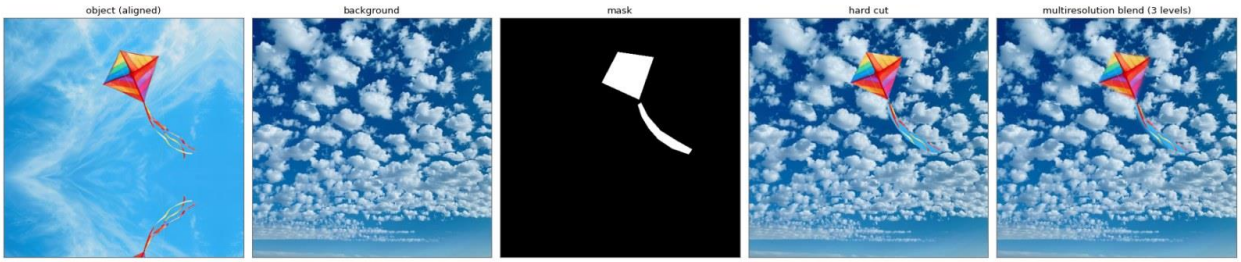


Figure 30. Kite in the sky: object (aligned), background, mask, hard cut, 3-level blend. The mask has two pieces, the kite body and the tail. In the hard cut the kite and its tail are placed on the dark blue clouds, but around the tail ribbons a rectangular patch

of the original, lighter blue sky comes along. The 3-level blend softens the kite's outline so that it sits naturally in the cloudy sky, and reduces, but does not remove, the light patch around the tail.

PARTIAL: the kite itself is convincing, but the tail mask is wider than the thin ribbons, so the light-blue sky of the source photo (a different tone from the dark background) shows as a halo. It cannot be hidden by blending because it is a genuine colour mismatch inside the mask; a tighter mask along the ribbons would fix it.

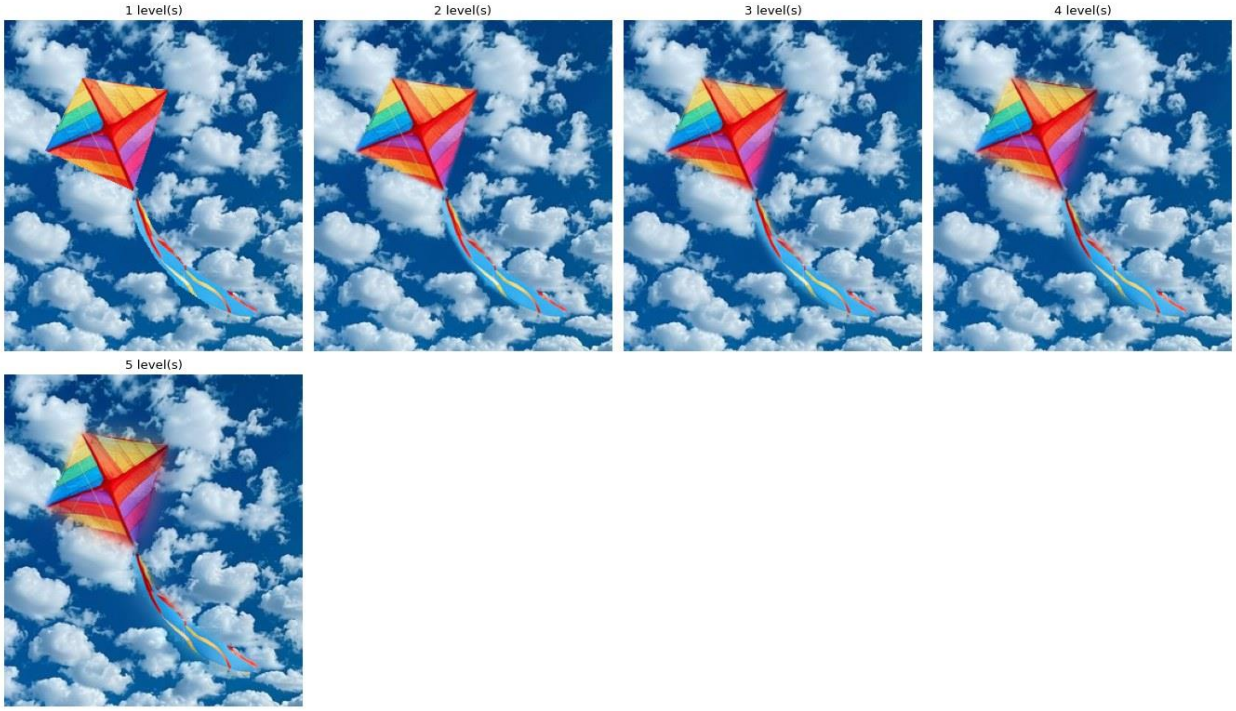


Figure 31. Choosing the number of levels for the kite: 1 to 5 levels (cropped to the kite). With 1–2 levels the kite is crisp but the light-blue patches around the ribbons are sharp-edged. With 4–5 levels the kite's red and orange bleed into the clouds as a coloured halo and the kite looks washed out. Three levels are the compromise: the kite edges are soft and its colours stay saturated.

INFO: a small pasted object needs few levels: "too many wash out a small pasted object", as the assignment warns.

> **Why blending band by band hides the seam that the hard cut shows.** A hard cut switches every frequency from one image to the other at the same pixel. Differences in the low frequencies (brightness, colour, shading) are then visible as a step along the cut, and the eye is very sensitive to such a step. Blending the whole image over a single wide window would hide that step but would also double-expose the fine texture (ghost edges), and a narrow window would keep the texture clean but leave the step. The Laplacian pyramid separates the image into frequency bands, and the Gaussian pyramid of the mask gives each band its own transition width: a wide one for the coarse bands (so colour and brightness fade smoothly over many pixels) and a narrow one for the fine bands (so edges and texture are never double-exposed). Every feature is blended over a distance matched to its own size, which is why no seam can be seen.

Part 3 — Seam Carving

Resizing a photo to a new aspect ratio normally means squashing everything or cropping things out. Seam carving instead removes, one by one, vertical seams of pixels that matter least. All the energies are computed with our own separable convolution (reflect padding), the seams are found with our own dynamic program, and horizontal seams (not needed here) would be found by transposing the image.

3.1 Finding and removing seams

Energy. The energy of a pixel is the gradient magnitude of 1.2 (finite differences `[1 0 −1]`), computed on the grayscale image with our filters. **Padding of the energy:** the border artifact of Part 1 would matter here. With zero padding the 1-pixel frame gets an energy of 0.459 (the interior average is 0.053), which makes the image borders look expensive; with reflect padding the frame energy is only 0.025. We therefore use reflect padding for the energy, and Figure 33 shows the difference in the seams.

Seam. A seam takes one pixel per row, and each pixel touches the one above it, directly or diagonally. **Dynamic program.** Let `M[i, j]` be the minimum total energy of a seam that ends at pixel `(i, j)`. The first row is `M[0, j] = e[0, j]`, and each further row is

```
M[i, j] = e[i, j] + min(M[i−1, j−1], M[i−1, j], M[i−1, j+1])
```

(neighbours outside the image are ignored), while a table `back[i, j]` remembers which of the three predecessors was the best. The seam ends at the smallest value of the last row, and is traced back row by row following `back`. The seam is then removed from all colour channels, the energy is recomputed, and the process repeats. Next to the image we keep an array holding each pixel's original column, which we cut with the same seams, so that every seam can be drawn where it sat in the original photo.

```
def cumulative_energy(energy):
    """M[i, j] = energy[i, j] + min(M[i-1, j-1], M[i-1, j], M[i-1, j+1])   (neighbors outside the image ignored).


    Also returns `back`, with back[i, j] in {-1, 0, +1}: the column offset of the best predecessor in row i-1.
    Ties are broken in the order  straight up, up-left, up-right  (so flat regions give straight seams).
    The loop runs over ROWS only; each row is processed for all columns at once.
    """
    energy = np.asarray(energy, dtype=np.float64)
    if energy.ndim != 2 or energy.size == 0:
        raise ValueError(f"energy must be a non-empty 2D array, got shape {energy.shape}")
    H, W = energy.shape
    M = energy.copy()
    back = np.zeros((H, W), dtype=np.int8)
    offsets = np.array([0, -1, 1], dtype=np.int8)
    cols = np.arange(W)
    for i in range(1, H):
        prev = M[i - 1]
        up_left = np.concatenate(([np.inf], prev[:-1]))        # predecessor at column j-1
        up_right = np.concatenate((prev[1:], [np.inf]))        # predecessor at column j+1
        candidates = np.stack([prev, up_left, up_right])       # (3, W)
        k = np.argmin(candidates, axis=0)                      # first minimum wins ties
        M[i] += candidates[k, cols]
        back[i] = offsets[k]
    return M, back


def find_vertical_seam(energy):
    """Cheapest top-to-bottom seam. Returns (seam, cost): seam[i] = column of the seam in row i."""
    M, back = cumulative_energy(energy)
    H, W = M.shape
    seam = np.empty(H, dtype=np.int64)
    j = int(np.argmin(M[-1]))                                  # cheapest end point in the bottom row (leftmost on ties)
    cost = float(M[-1, j])
    seam[-1] = j
    for i in range(H - 1, 0, -1):                              # trace back, one row at a time
        j += int(back[i, j])
        seam[i - 1] = j
    return seam, cost
```

Why trying every possible seam would take forever

A seam chooses one of 2 or 3 columns in each of the H − 1 following rows, so for a 484×714 image there are between W·2^(H−1) and W·3^(H−1) seams, **between 10^148 and 10^233**, far more than the atoms in the universe (about 10^80). The dynamic program does not enumerate seams: it keeps only the best cost per pixel, because the cheapest seam through a pixel must extend the cheapest seam ending at one of its three upper neighbours. Each pixel needs 3 comparisons, so a whole seam costs **H·W·3 = 1 036 728 comparisons** instead of 10^148, and we run it once per removed seam (about 0.13 s per seam including the energy recomputation in our run).



Figure 32. Seam carving of the Broadway Tower: the energy (left) and the 214 removed seams drawn in red on the original (right). The energy (finite-difference gradient magnitude) is bright on the tower's stones and outlines, on the walker, on the horizon and along the cloud streaks, and almost black in the clear sky. The 214 seams (30 % of the 714 columns) all run through the sky and the grass, avoid the tower and the walker completely, and bunch up in the sky where the energy is lowest. The thick red band at the right edge is the sky on the right of the tower.

SUCCESS: seams keep to the low-energy regions, so the content (tower, walker) is untouched while 30 % of the width disappears. The seams' average energy rose from 0.0084 (first seam) to 0.0188 (last seam), against an image average of 0.0530: even the last seam was 2.8× cheaper than an average pixel.



Figure 33. Effect of the padding on the energy: zero padding (left) versus reflect padding (right, used). With zero padding the frame of the image carries a false high energy, so seams avoid the borders (1 717 of 103 576 seam pixels fall in the 5 outermost columns on each side); with reflect padding they use them more naturally (2 565), and the mean seam energy is also lower (0.0148 versus 0.0166).

INFO: reflect padding avoids the border artifact explained in 1.2.

Removing the 214 seams took 27.8 s on our machine (130 ms per seam including the energy recomputation).

3.2 Results and comparisons

We reduced the width of **six images by about 30 %**: `rocket.jpg`, `home.jpg` and `starry_night.jpg` from the starter pack, and three of our own (a person on a beach, a sunset landscape, and a building façade), plus the Broadway Tower used as the running example. For each image we compare, at the same final width: normal

resizing (squeeze), a center crop, and seam carving with two energy functions: the **finite differences of 1.2** and the **DoG of 1.3 (σ = 2)**. For every image we show the original and its four narrower versions, and, for each image, the energies and the seams that each energy function picks.

Image	Size → width	Seams	Mean seam energy ÷ image mean: FD (last seam)	DoG (last seam)	FD/DoG overlap (Jaccard)	Result
Broadway Tower	714×484 → 500	214	0.28 (0.35)	0.31 (0.42)	0.56	success
rocket.jpg	640×427 → 448	192	0.13 (0.20)	0.22 (0.29)	0.43	success
home.jpg	512×384 → 358	154	0.15 (0.33)	0.21 (0.36)	0.53	success
starry_night.jpg	720×568 → 504	216	0.45 (0.54)	0.40 (0.50)	0.36	**failure**
own: person on beach	612×407 → 428	184	0.58 (0.66)	0.55 (0.61)	0.34	DoG success, FD flawed
own: sunset landscape	626×468 → 438	188	0.63 (0.70)	0.69 (0.76)	0.32	all methods fine
own: building façade	1000×562 → 700	300	0.27 (0.35)	0.30 (0.39)	0.59	**failure**

The ratio “mean seam energy ÷ image mean energy” says how much structure the removed seams had to cross (low = lots of free, flat space). The Jaccard overlap is the share of removed pixels that both energies agree on. A low ratio does not guarantee a good result (the façade has a low ratio and fails) and a high ratio does not mean a bad one (the sunset has a high ratio but is untroubled), because what matters is what the removed pixels belonged to.

Broadway Tower (the example image)



Figure 34. Broadway Tower, width 714 → 500: original, resized, center crop, seam carved with finite differences, seam carved with DoG (σ = 2). Resizing squeezes the whole picture, so the tower and the walker become 30 % thinner. The center crop keeps the tower’s proportions but cuts the walker out and clips the right tower at the frame. Both seam-carved versions keep the tower and the walker at their original proportions by removing sky and grass.

SUCCESS: the empty sky and grass can be shrunk without anyone noticing; the objects keep their shape.

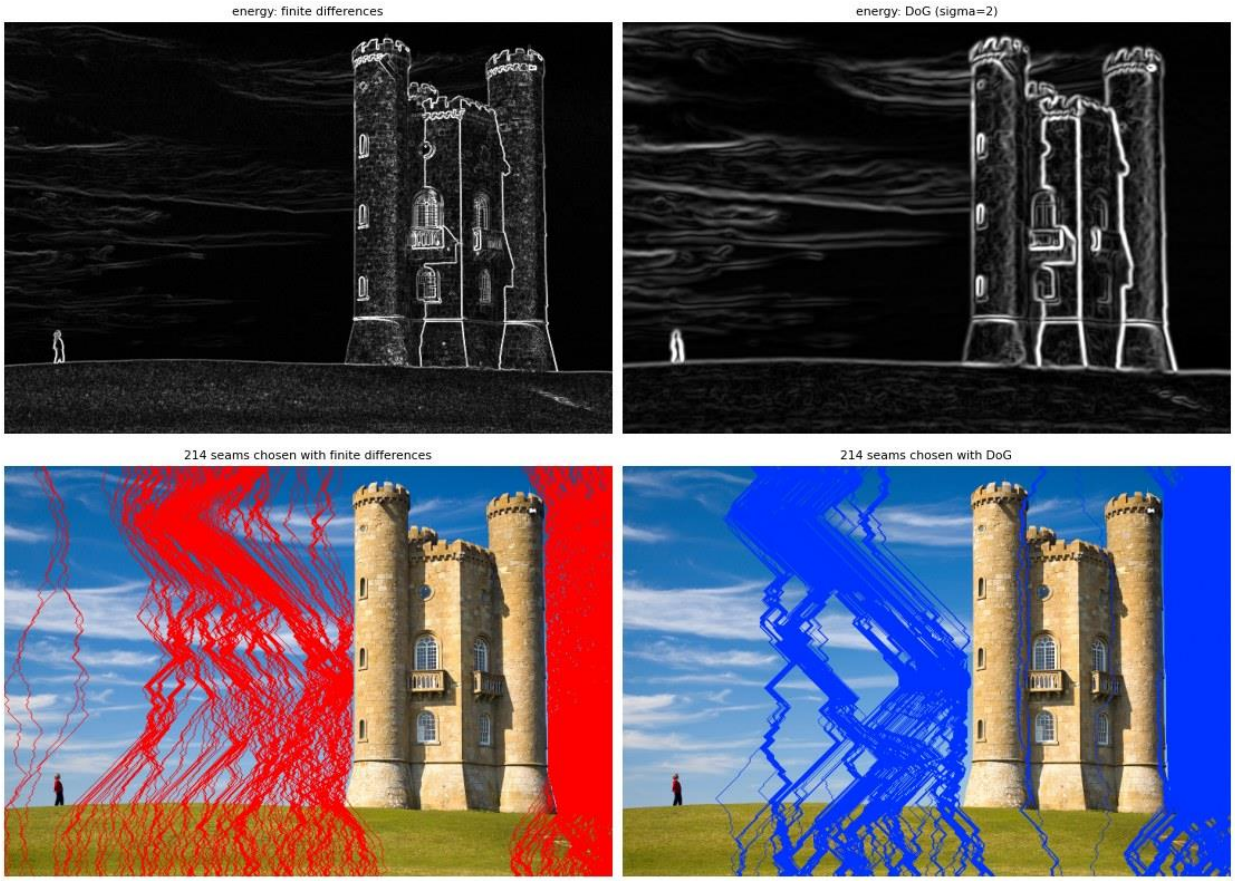


Figure 35. Broadway Tower: the energy of each function (top) and the 214 seams each picks (bottom: finite differences in red, DoG in blue). Both energies protect the tower and the walker. The FD seams are thin and wander across the grass texture, because the finite-difference energy responds to every blade of grass; the DoG seams form thicker, smoother bundles that run straight down the sky and keep a margin from the tower, because the blur turns the energy into broad bands around edges and removes the small grass texture. The overlap (Jaccard index) of the pixels removed by the two energies is 0.56.

rocket.jpg (starter pack)



Figure 36. Rocket, width 640 → 448. Resizing makes the rocket and its masts noticeably slimmer. The center crop loses most of the two large lattice towers at the sides of the picture. Both carved versions keep the rocket, the two thin masts and the lattice towers at their natural width, and compress the empty dark sky between them, so the towers end up closer to the rocket.

SUCCESS: the sky is flat (the mean seam energy is only 0.13 times the image average), so almost nothing of value has to be removed. The only visible blemish is a slight bend at the base of the right mast in the DoG version.

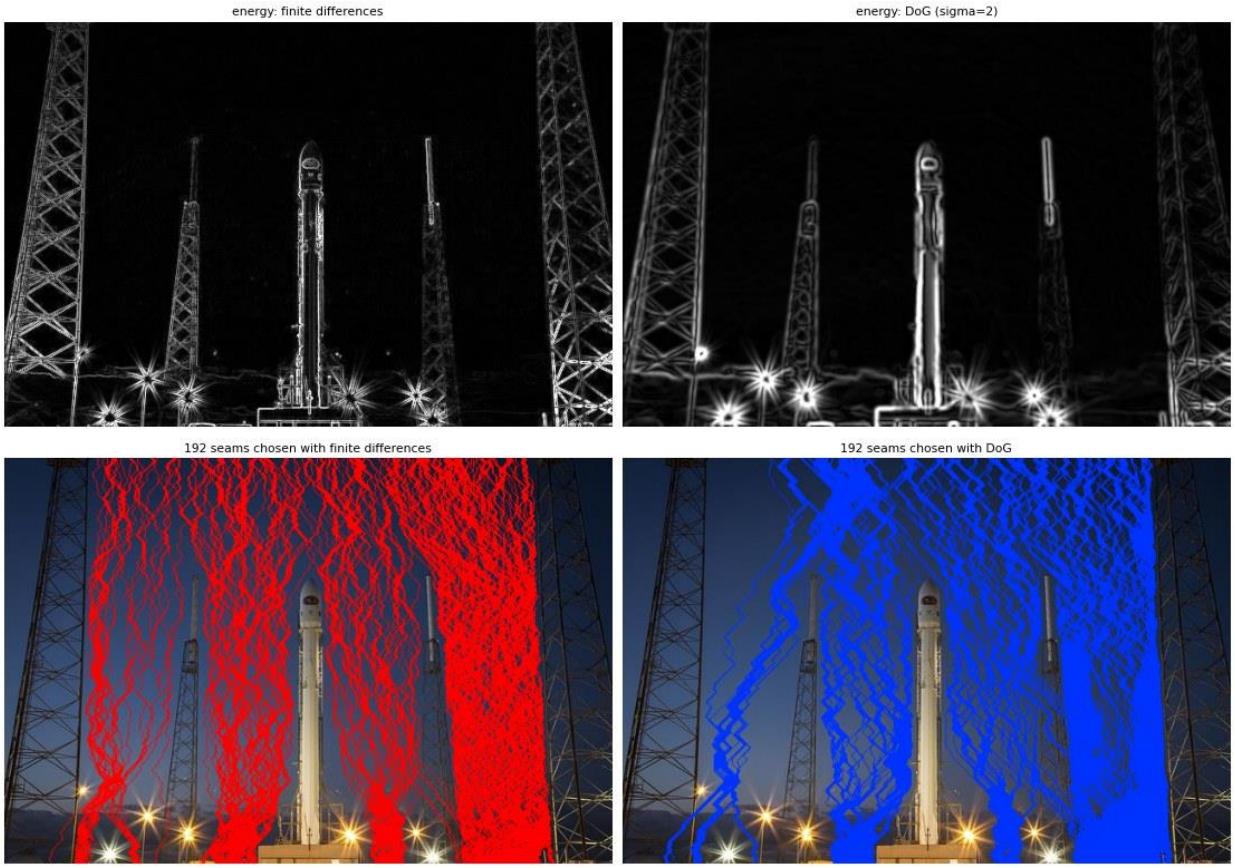


Figure 37. Rocket: energies and the 192 seams picked by finite differences (red) and by DoG (blue). The finite-difference seams are many thin lines spread over the whole sky, and the DoG seams gather into two thick bands, one to the left of the rocket and one between the rocket and the right tower, because the DoG energy has wide halos around the rocket, the masts and the star-shaped lights that seams avoid. The overlap (Jaccard index) between the two removals is only 0.43.

home.jpg (starter pack)



Figure 38. Home, width 512 → 358. Resizing squeezes the building: the windows and ornaments become visibly narrower. The center crop discards the strips at the left and right of the photo, including part of the building. Both carved versions keep the building's ornaments, windows and towers at natural width and remove blue sky, mainly on the left; some of the small birds are lost or moved closer together.

SUCCESS: the large uniform sky provides cheap seams (mean seam energy 0.15 times the image average) and the building is in the part of the image that is not touched.

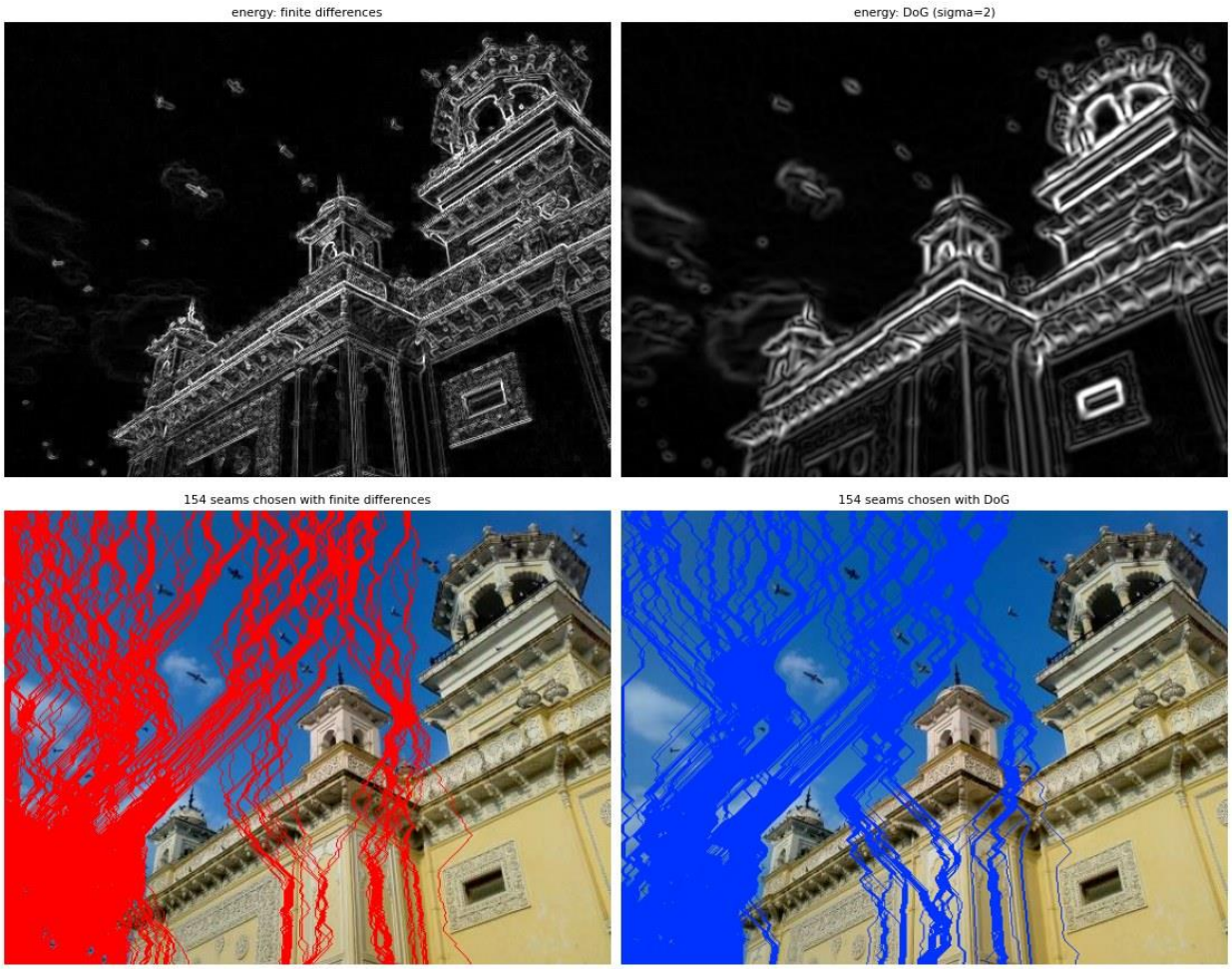


Figure 39. Home: energies and the 154 seams picked by finite differences (red) and by DoG (blue). Both energy functions send most seams through the sky on the left and down along the left side of the picture, where there are few strong edges; a few bundles also pass through plain stucco areas of the building between the towers (thin lines for finite differences, thicker bundles for DoG), and none crosses the ornaments or the windows.

starry_night.jpg (starter pack): a failure case



Figure 40. Starry Night, width 720 → 504. Resized and cropped images are acceptable in their own ways (the resized one is uniformly squeezed, the crop clips the moon at the right edge). The carved versions narrow the cypress tree on the left: with finite differences it is slimmer but recognisable, with DoG it is reduced to a thin sliver.

FAILURE: seam carving damages the picture's main subject (see the analysis below).



Figure 41. Starry Night: the cypress under uniform resizing and under the two seam-carving energies. Top: the original and the uniformly squeezed version. Bottom: seam carving with finite differences (cypress slimmer but recognisable) and with DoG (cypress reduced to a sliver, its upper part almost gone).

FAILURE: the cypress is a large, dark region that has little gradient energy, because it is smooth compared with the swirling brush strokes everywhere else; the sky around it has strong contrast and strong texture in every pixel. For both energies the cheapest vertical seams run through the cypress. Measured on the cypress (pixels darker than 0.22 in the left 330 columns, 14.7 % of the image): finite differences remove 54.0 % of its pixels and DoG removes 64.4 %, where a uniform 30 % resize would remove 30 %; 26.3 % (FD) and 31.4 % (DoG) of all removed pixels lie on the cypress. At row 350 the cypress is 75 px wide and becomes 21 px (FD) or **1 px** (DoG), at row 500 it goes from 197 px to 99 px (FD) or 77 px (DoG). The DoG energy is worse because its blur flattens the fine brush strokes inside the cypress, so the cypress looks even more “empty” to it. In addition, there are no truly flat regions in this painting: the mean seam energy is 0.45 times the image average, the highest of the starter images, so every seam has to cross strokes and cannot find a free path.

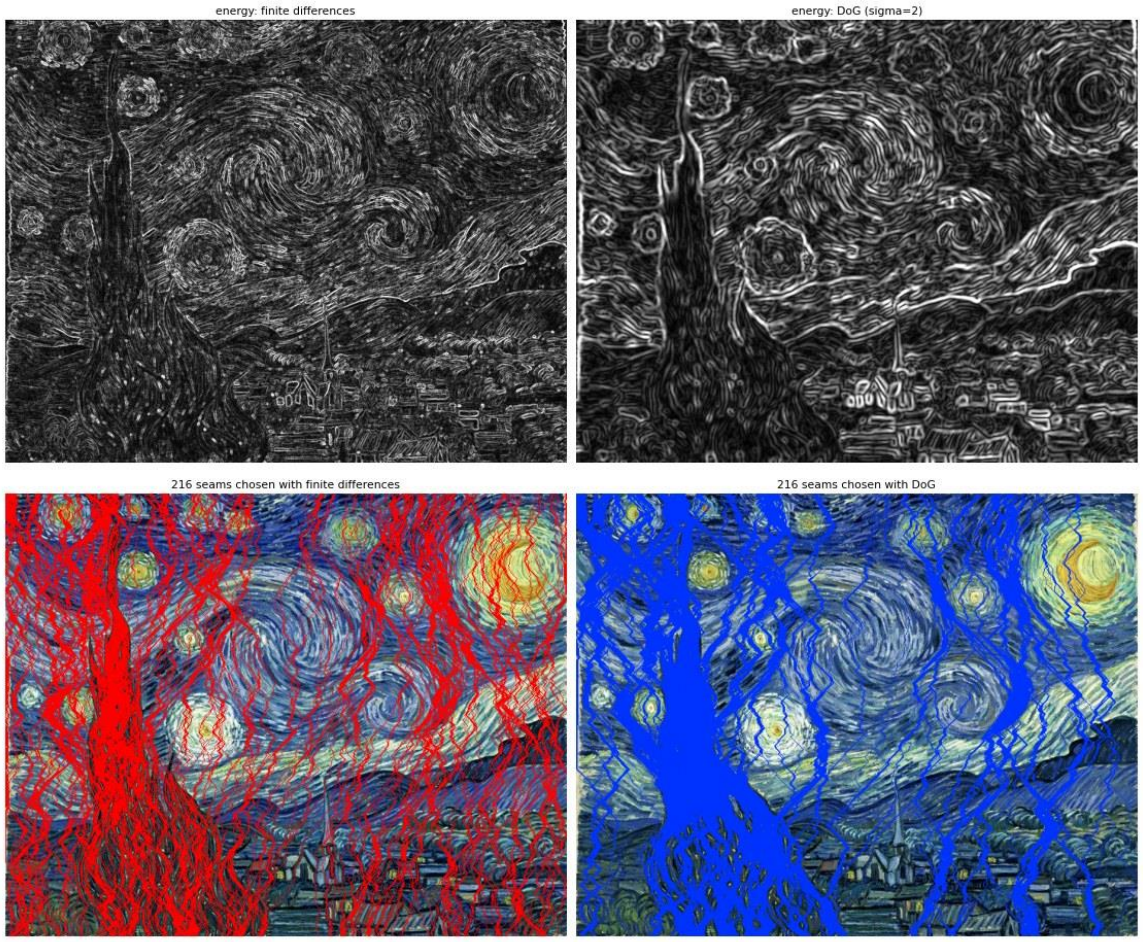


Figure 42. Starry Night: energies and seams of the two functions. The energy is high over most of the painting because every brush stroke is an edge, which leaves very few cheap paths. In both maps the cypress is darker (cheaper) than the swirling sky around it, and in the DoG map its interior is almost black. In the bottom row the seams of both functions run in dense bundles straight through the cypress (26.3 % of all removed pixels for finite differences, 31.4 % for DoG).

Own image: a person on a beach



Figure 43. Person on a beach, width 612 → 428. Resizing makes the man uniformly narrower. The center crop keeps his proportions (the beach around him is simply narrower). With finite differences the man is visibly thinner, with a pinched torso and wavy legs; with DoG he keeps his proportions, as in the crop.

PARTIAL: DoG energy succeeds, FD energy partially fails, for the reason given below the next figure.



Figure 44. Zoom on the man for the four narrow versions (resized, center crop, seam carved with FD, seam carved with DoG). The same 220×307 pixel window, enlarged 1.6×, around the man. The finite-difference result is clearly slimmer than the crop and the DoG results: his shirt is narrower and his legs are distorted.

PARTIAL: see the explanation below.

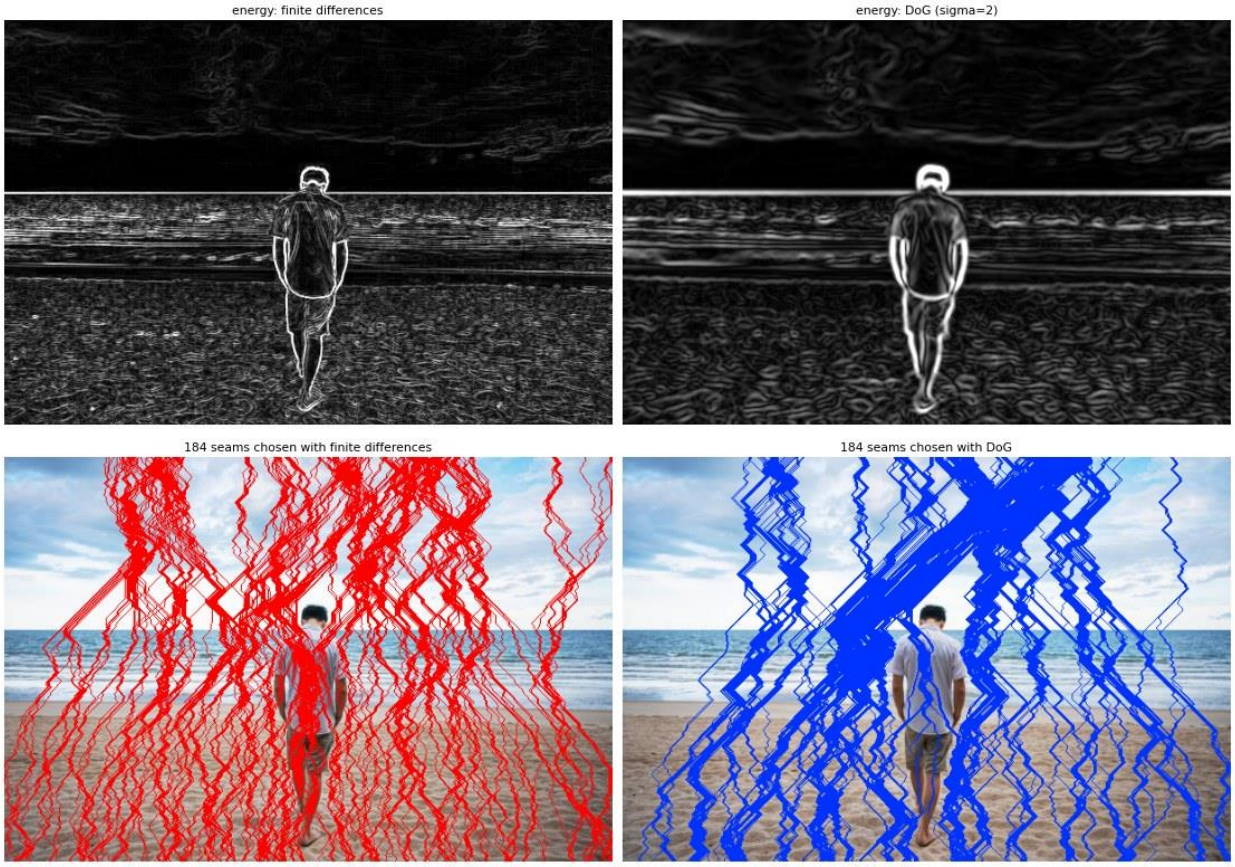


Figure 45. Person on a beach: energies and the 184 seams of each function. The finite-difference energy (top left) is high everywhere on the sand and the sea (fine texture), and the man appears mostly as thin outlines with a dark, low-energy interior (the white shirt is flat). Seams therefore find free columns through the man's body and also between texture pixels. The DoG energy (top right) is smoother and weaker on the sand texture, so the sand is comparatively cheap, and the man's outline becomes a thick, strong band; the DoG seams (blue) go around him through the sand, the sky and the sea.

PARTIAL: the blur of the DoG makes the objects stand out against the texture and protects the person, whereas the finite differences see texture and object alike and pass seams through the body.

Own image: sunset landscape



Figure 46. Sunset beach, width 626 → 438. All four narrower versions look natural. The squeezed version has a slightly compressed sun and wave pattern, the crop is the most direct, and both carved versions keep the sun round and the horizon straight.

SUCCESS: there is no object whose proportions matter (smooth sky, sea and sand), so any resizing method works; seam carving gains little here. The ratio of seam energy to image energy is high (0.63 FD, 0.69 DoG) because the sea and sand are textured, but the removed pixels belong to those textures, where distortion is invisible.

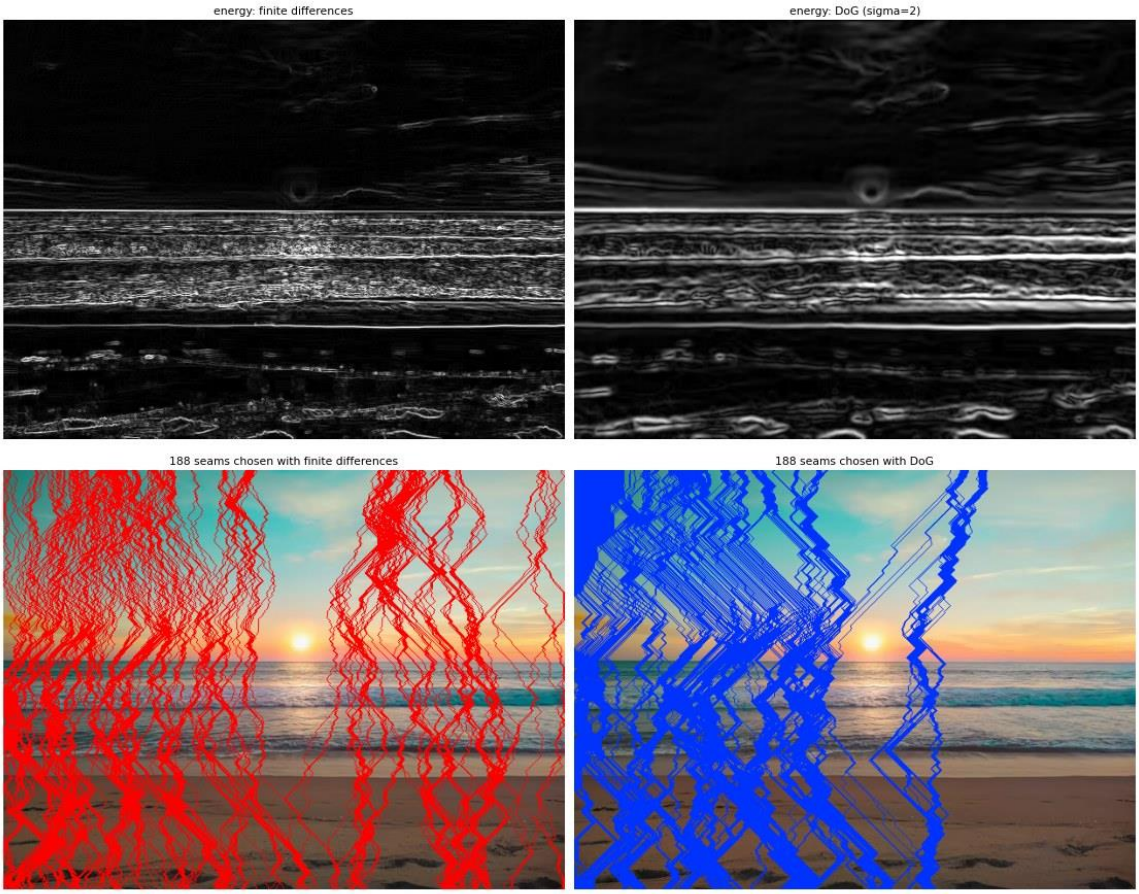


Figure 47. Sunset beach: energies and seams of each function. The strong energy is concentrated on the horizon and the surf lines. The seams of both functions run through the sky and sand, avoiding the sun and crossing the horizon where the colours are smooth.

Own image: building façade, our failure case



Figure 48. Building façade, width 1000 → 700. The photo shows three façades (pink, green, blue) with regular grids of windows and straight horizontal ledges. Resizing keeps all straight lines straight but makes every window 30 % narrower. The center crop keeps the windows' proportions but cuts off parts of the red and blue buildings. The carved versions keep the windows at their natural width, but the grid is broken: the windows are closer together and some window frames and ledges show steps and kinks.

FAILURE: seam carving destroys the regularity of the façade (details below).



Figure 49. Zoom on the left of the façade (resized, seam carved with FD, seam carved with DoG). In the resized version the white ledge, the window frames and the border between the pink and green walls are all straight. In both seam-carved versions the white ledge is broken into pieces with small steps, the window frames are chopped differently from row to row, and the border between the two walls is a jagged staircase.

FAILURE: every vertical seam is only allowed to move one pixel per row, but it has to pass through the horizontal ledges and cross the walls between the windows. The seams (next figure) concentrate in the flat wall strips between the window columns because these are cheap, and are removed there row by row. The seam cannot follow a straight vertical line and also avoid the ledges, so each ledge is cut at different positions in different rows, and the straight horizontal and vertical lines are bent into small zig-zags that the eye notices immediately because it knows how straight lines should look.

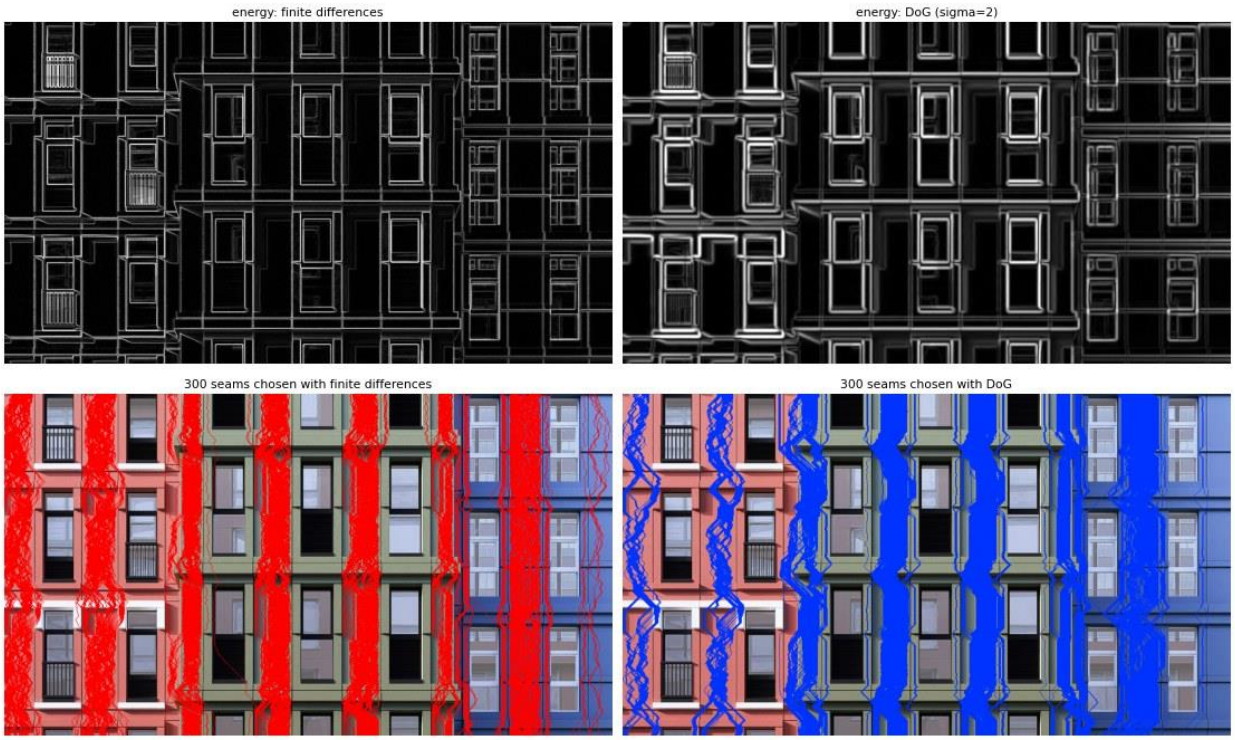


Figure 50. Façade: energies and the 300 seams of each function. The energy has strong straight lines (window frames, ledges). Almost all seams of both functions are in the flat wall between the window columns, in dense vertical bundles. Both energy functions choose almost the same removal (59 % overlap): the problem is not the choice of energy, but the structure of the image, a regular grid of straight lines, which no energy that treats pixels independently can protect.

FAILURE: seam carving fails on images with strong, regular, straight structure where any bend is visible; removing 30 % of the width means removing about one third of the wall between the windows, row by row, in an inconsistent way. A pure resize or a crop would be better here.

Summary of 3.2: how the seams of the two energy functions differ and when seam carving fails. **Finite differences** respond to every intensity change, including texture and noise, so the energy is “peppered” and seams are thin, wandering and numerous in textured areas; thin objects with low-contrast interiors (a white shirt) can be traversed. **DoG (σ = 2)** is the same energy blurred: edges become wide bands and weak texture disappears, so the seams are thicker, smoother, run straighter and keep a margin from edges, which protects people and large objects, but fine structure that is smaller than σ (brush strokes inside the cypress) is blurred into “nothing” and is not protected. Neither energy is best everywhere: DoG wins on the person, FD does better on Starry Night’s cypress. **Seam carving fails** when (a) the picture has no flat regions, so every seam must cross texture (Starry Night), or (b) the content has regular straight geometry that must remain straight (façade), or (c) the main object is itself low-energy (flat interior). It is worth its cost on photos with large empty or smooth areas and compact objects (tower, rocket, building against the sky, and, with DoG, the person).

Bonus — Object removal and enlargement by seams

We removed an object from a photo of our own and then grew the photo back to its original width with inserted seams. The photo (612×459) shows a plastic water bottle standing on a wooden table in front of a blurred forest.

Step 1 — removing the object

- **Mask.** Drawn in code with `cv2.fillPoly` around the bottle and then grown by three iterations of a 3×3 dilation (about 3 px), so that the blue cap and the bright streak at the shadow's edge are covered too. The mask has 39 886 pixels and its widest row is 125 px.
- **Energy.** The finite-difference energy in `float64`, set to -10^6 on the masked pixels, so that the cheapest seam always passes through the mask.
- **Loop.** Find the cheapest seam, remove it from the photo and from the mask, repeat until no masked pixel is left. This needed **k = 125 seams** (14.4 s), which reduced the photo to width 487.

Step 2 — growing the photo back by k seams

- Make a copy of the reduced photo and remove k = 125 seams from the copy (the copy is only used to find them), keeping an array that gives each pixel's column in the reduced photo, so that we know where each seam sat in the reduced photo.
- Widen the reduced photo row by row: every pixel is written once, but each pixel of the k seams twice, the first copy averaged with its left neighbour and the second with its right neighbour; at the border the pixel itself is used as the missing neighbour. This took 10.7 s and gave width 612.

```
def insert_seams(image, seams):
    """Widen `image` by k = len(seams) columns: every pixel is written once, but each pixel on a seam is written TWICE --
    the first copy averaged with its LEFT neighbor, the second with its RIGHT neighbor (a missing neighbor at the border
    is replaced by the pixel itself).  Neighbors are taken from `image`, not from the output.


    seams: (k, H) integer array, columns in `image`; no pixel may be used by two seams.
    Returns (out, src, inserted): the widened image, src[i, j] = column of `image` that out[i, j] comes from, and a boolean
    mask of the pixels that belong to inserted seams (both copies).
    """
    image = np.asarray(image, dtype=np.float64)
    H, W = image.shape[:2]
    seams = np.asarray(seams, dtype=np.int64).reshape(-1, H)
    k = seams.shape[0]
    if seams.size and (seams.min() < 0 or seams.max() >= W):
        raise ValueError("seam column outside the image")
    on_seam = np.zeros((H, W), dtype=bool)
    for s in seams:
        if on_seam[np.arange(H), s].any():
            raise ValueError("two seams use the same pixel")
        on_seam[np.arange(H), s] = True
    reps = 1 + on_seam.astype(np.int64)                                         # 2 for seam pixels, else 1
    rows = np.arange(H)[:, None]
    src = np.repeat(np.tile(np.arange(W), (H, 1)).ravel(), reps.ravel()).reshape(H, W + k)
    inserted = np.repeat(on_seam.ravel(), reps.ravel()).reshape(H, W + k)
    second = np.zeros((H, W + k), dtype=bool)
    second[:, 1:] = inserted[:, 1:] & (src[:, 1:] == src[:, :-1])               # the second of a pair of identical sources
    first = inserted & ~second
    left, right = np.maximum(src - 1, 0), np.minimum(src + 1, W - 1)            # missing neighbor -> the pixel itself
    pix = image[rows, src]
    pick = lambda a: a[..., None] if image.ndim == 3 else a
    out = np.where(pick(first), 0.5 * (pix + image[rows, left]),
                   np.where(pick(second), 0.5 * (pix + image[rows, right]), pix))
    return out, src, inserted


def enlarge_by_seams(reduced, k, energy_fn=energy_fd):
    """Grow `reduced` back by k columns.  The k seams are found ONCE, by removing k seams from a COPY of the photo (the copy
    is only used to find them; the coordinate tracking of carve() tells where each seam sat in `reduced`), then all k are
    inserted at the same time.  Returns dict(image, seams, inserted)."""
    found = carve(reduced, k, energy_fn)                                        # works on a copy; `reduced` is untouched
    out, src, inserted = insert_seams(reduced, found["seams"])
    return dict(image=out, seams=found["seams"], inserted=inserted, src=src)
```

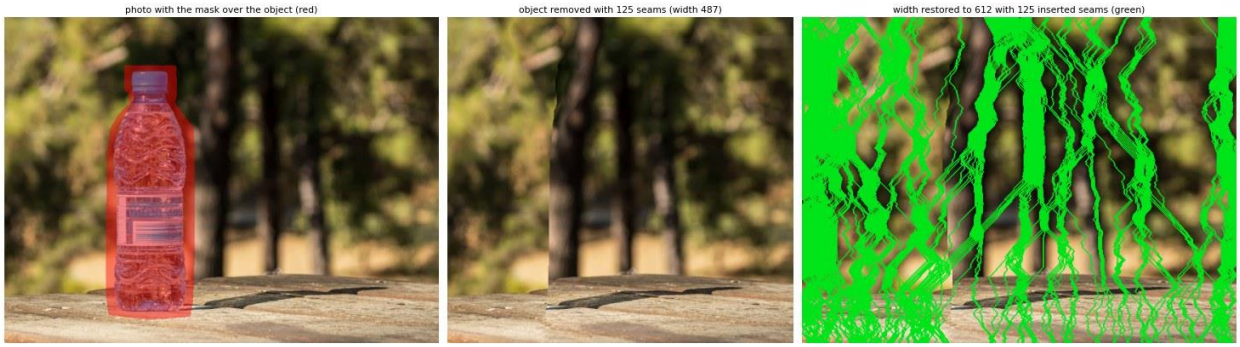


Figure 51. Object removal and restoration of the photo's width: the photo with the mask over the bottle (red), the photo without the bottle (width 487), and the photo back at width 612 with the 125 inserted seams drawn in green. Left: the mask covers the whole bottle and a margin around it. Middle: with 125 seams removed through the mask the bottle has disappeared and the table and the forest close up over the place where it stood. Right: 125 new seams (green) are inserted where the seams are cheapest (spread over the blurred background and the table, with dense bundles along the tree trunks), restoring the original width.

SUCCESS: the bottle is gone and the photo is back at its original size. A flaw remains: a faint vertical step at the place where the bottle stood (visible on the left of the trunk and as a small offset in the table surface), see the zoom.



Figure 52. Zoom around the removed object: the 125 removal seams on the original (red), the original, the object removed, and the restored width (same window). The removal seams form a solid red column exactly on the bottle, so only the bottle's columns were removed. After removal the table's edge and the trunk next to the bottle's position are joined along a vertical line; the restored image keeps that line, while the rest of the picture looks natural.

PARTIAL: the join between the two sides of the removed object cannot be perfectly hidden, since the seams had to pass through the bottle (a hard constraint) rather than where the energy is low.

Why the k seams come from a copy, not from inserting the cheapest seam k times

If we inserted the single cheapest seam, recomputed the energy, and repeated, the next cheapest seam would almost always be the neighbour of the one we just inserted: a newly inserted seam is an average of its neighbours, so it has nearly zero gradient and is the cheapest place in the image again. The same region would be stretched over and over, producing wide smears. Finding all k seams on a copy by removal gives k different, non-overlapping seams spread over the image, each one cheap in its own place, so each pixel is duplicated at most once. Our measurements:

	k seams found on a copy (ours)	cheapest seam inserted k times
Maximum number of times a pixel of the reduced photo appears in the result	2	up to 126
Pixels written twice / more than twice	57 375 / 0	1 658 duplicated / 1 556 more than twice
Distinct columns of the reduced photo that are stretched (mean over rows, of k = 125)	125.0	3.6

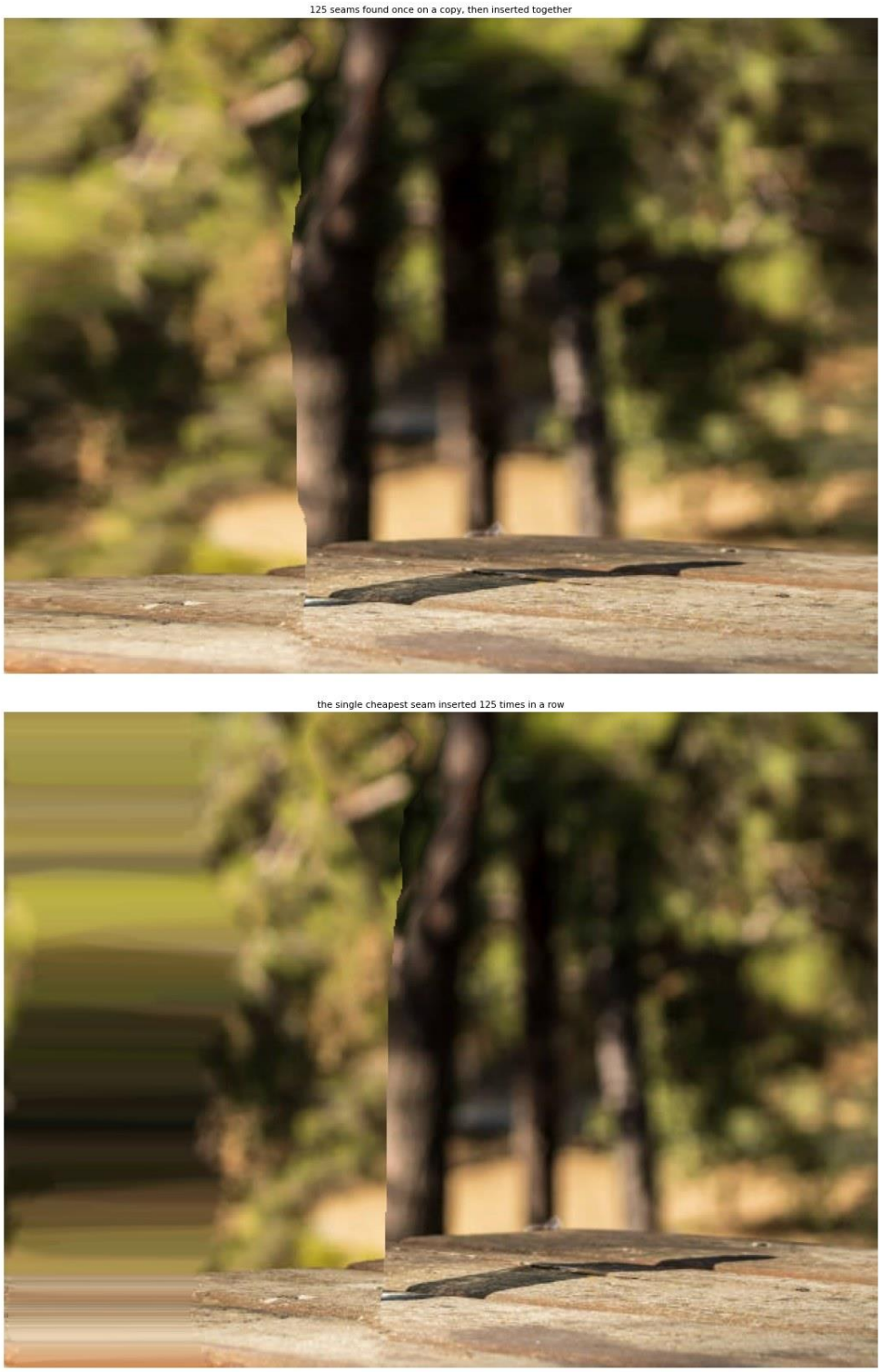


Figure 53. Comparison of the two ways of widening the photo: k seams found once on a copy (top) and the cheapest seam inserted 125 times (bottom). Top: the content is stretched evenly, with 125 different seams each used once. Bottom: the same column region is stretched again and again, so the left part of the photo turns into horizontal smears (streaks of colour) and a very wide band of stretched pixels appears at the left edge.

SUCCESS: the copy-based method gives a natural widening; repeated insertion of the cheapest seam gives visible artifacts, as the numbers above predicted (3.6 distinct columns stretched instead of 125).

Appendix A — Code organisation and how to reproduce

File (in code/)	Content
conv.py	1.1: loop, vectorized and separable convolution, padding
filters.py, edges.py	Gaussian / box / DoG kernels, finite differences, gradient magnitude, edge detection (1.2, 1.3)
freq.py	Hybrid images, Fourier magnitude, aliasing experiment (2.1, 2.2)
pyramids.py	reduce, expand, Gaussian and Laplacian pyramids, rebuild (2.3)
blend.py	Multiresolution blending (2.4)
seam.py	Energy functions, dynamic program, seam removal and insertion, object removal (3.1, 3.2, Bonus)
run_*.py	One script per experiment, writing figures to results/ and logs to results/logs/
run_own.py, Assignment1_run.ipynb	Own-image hybrids, blends, seam comparisons and bonus; the notebook contains a complete run with its outputs
tests/	84 automatic tests: convolution (20), edges (9), frequency (9), aliasing (4), pyramids (12), blending (8), seams (15), bonus (7); all pass

Everything runs from the code/ folder with python run_all.py (the automatic tests plus all starter-pack experiments; --tests-only and --skip-benchmark are available) and python run_own.py for our own images.

Appendix B — Important run logs

The logs are reproduced as written by the scripts (lines are wrapped by the page width). Logs for our own images come from the full run saved in `Assignment1_run.ipynb`.

1.1 — sanity check and timing (`run_1_1_benchmark.txt`)

```
SANITY CHECK vs scipy.signal.convolve2d(mode='same'), 300 random cases
  kernels used: {'separable kernels': 157, 'non-separable kernels': 143}
  max |ours - scipy|  loop       : 3.55e-14
  max |ours - scipy|  vectorized : 3.55e-14
  max |ours - scipy|  separable  : 2.13e-14
TIMING on a 256x256 image (seconds; vectorized/separable = best of several repeats)
   size       loop   vectorized   separable  loop/vec  vec/sep
  3x3       0.7407      0.00103     0.00255       719      0.4
  5x5       1.8229      0.00236     0.00211       774      1.1
  7x7       3.4207      0.00553     0.00322       619      1.7
  9x9       6.0423      0.00825     0.00440       732      1.9
 11x11      8.3810      0.00995     0.00324       842      3.1
 13x13     11.1958      0.01732     0.00333       646      5.2
 15x15     16.0649      0.01936     0.00377       830      5.1
 17x17     19.9286      0.02566     0.00520       777      4.9
 19x19     27.8159      0.03082     0.00472       903      6.5
 21x21     31.3197      0.03849     0.00387       814     10.0
 23x23     40.3912      0.05512     0.01379       733      4.0
 25x25     45.9935      0.06299     0.00501       730     12.6
 27x27     48.9960      0.06220     0.00740       788      8.4
 29x29     58.3853      0.08721     0.00760       669     11.5
 31x31     66.3780      0.09805     0.00753       677     13.0
saved results/benchmark_1_1.csv and results/benchmark_1_1.png
```

1.2 and 1.3 — edges, threshold sweep, noise levels (`run_1_2_1_3.txt`)

```
image: part1_filters/facade.png  shape=(670, 736)
[1.2] finite differences, zero padding
  gx range [-0.996, 0.988]  gy range [-0.918, 0.941]  mag max 1.232
  display scale: derivatives +-0.514 (99.5th pct of interior |g|), magnitude 0..0.626; arrays not modified
  threshold 0.25: 9.4% of all pixels, 8.9% of interior pixels are edges
  robust noise level of gx (MAD): 0.029  -> threshold is 8.6 x that level
  frame: mean magnitude on the 1-px border = 0.478 (zero pad) vs 0.041 (reflect); interior 0.090
         border pixels flagged as edges: 95.7% (zero pad) vs 1.8% (reflect)
         top row: corr(magnitude, image brightness) = 0.63
THRESHOLD SWEEP (share of pixels flagged as edges: whole image / interior)
  1.2  t=0.15:  18.6% /  18.0%
  1.2  t=0.25:   9.4% /   8.7%
  1.2  t=0.35:   4.8% /   4.3%
  1.2  t=0.45:   2.4% /   2.0%
  1.3  t=0.05:  24.3% /  22.6%
  1.3  t=0.08:  11.6% /  10.2%
  1.3  t=0.11:   5.7% /   4.6%
  1.3  t=0.15:   2.1% /   1.5%
[1.3] DoG, sigma = 2, filter size 13x13
  DoG_x: min -0.0222 max 0.0222 sum -2.8e-16; rank-1 (separable): True
  DoG_y == DoG_x^T: True
  gx range [-0.364, 0.365]  mag max 0.365
  display scale: derivatives +-0.158, magnitude 0..0.182
  threshold 0.08: 11.6% of all pixels, 10.3% of interior pixels are edges
  1.2 threshold 0.25 keeps 8.8% of the same interior region (similar density -> fair comparison)
  robust noise level of DoG gx (MAD): 0.013  (finite difference: 0.029)
  magnitude 99th percentile (interior): 1.2 = 0.541, DoG = 0.163
  frame band width on a constant image: 1.2 -> 1 px per side, DoG -> 6 px per side (the 13x13 filter reaches 6 px)
figures written to results/
```

2.1 — σ sweep for man + cat (`run_2_1_sigma_sweep.txt`)

```
sigma -> half-amplitude cutoff of the Gaussian (cycles/pixel, period in pixels)
  sigma 2: 0.0937 cyc/px (period 10.7 px)
  sigma 3: 0.0625 cyc/px (period 16.0 px)
  sigma 4: 0.0468 cyc/px (period 21.3 px)
  sigma 6: 0.0312 cyc/px (period 32.0 px)
  sigma 9: 0.0208 cyc/px (period 48.0 px)
near = correlation of the hybrid's fine-detail band with the CAT's (want high) and the MAN's (want low)
far  = RMSE between the hybrid and the MAN / the CAT after shrinking to 1/8 with our reduce (want low / high)
s_low s_high | near~cat near~man | far RMSE man far RMSE cat
    4      2 |    0.959    0.040 |       0.0079       0.2633
    4      3 |    0.982    0.037 |       0.0093       0.2627
    4      4 |    0.990    0.036 |       0.0116       0.2620
    4      5 |    0.993    0.035 |       0.0144       0.2613
    6      2 |    0.962    0.016 |       0.0141       0.2619
    6      3 |    0.985    0.015 |       0.0150       0.2613
    6      4 |    0.993    0.015 |       0.0165       0.2606
    6      5 |    0.996    0.015 |       0.0186       0.2599
    8      2 |    0.963    0.009 |       0.0206       0.2603
    8      3 |    0.986    0.008 |       0.0212       0.2597
    8      4 |    0.993    0.008 |       0.0223       0.2590
    8      5 |    0.996    0.008 |       0.0239       0.2583
FOURIER CROSS of the low-pass man (sigma=6): log-magnitude on the central axes minus just beside them
  plain image                : +3.81
  image x Hann window (no border jump): +0.36
```

2.1 — chosen σ values and clipping for the three hybrids

```
[man_cat] sigma_low=6 (half-amplitude cutoff 0.0312 cyc/px), sigma_high=4 (cutoff 0.0468 cyc/px)
[man_cat] hybrid range [-0.205, 1.277]; 1.47% of pixels clipped for display/saving
[own1] sigma_low=8 (half-amplitude cutoff 0.0234 cyc/px), sigma_high=3 (cutoff 0.0625 cyc/px)
[own1] hybrid range [-0.066, 0.981]; 0.01% of pixels clipped for display/saving
[own2] sigma_low=6 (half-amplitude cutoff 0.0312 cyc/px), sigma_high=3 (cutoff 0.0625 cyc/px)
[own2] hybrid range [-0.151, 1.453]; 17.49% of pixels clipped for display/saving
```

2.2 — aliasing experiment (`run_2_2_aliasing.txt`)

```
original (512, 512) -> shrunk (128, 128); region for the false-circle measure: 8284 px, where the ideal result has strength 0.1187
EXPERIMENT: strength of the false pattern (1.00 = full-amplitude rings, 0 = flat gray) and RMSE to the ideal result
 sigma  strength  RMSE vs ideal
  none     0.999         0.2873
   0.5     0.836         0.2453
  0.75     0.626         0.1916
   1.0     0.446         0.1483
  1.25     0.297         0.1181
   1.5     0.187         0.1060
  1.75     0.113         0.1085
   2.0     0.068         0.1182
  2.25     0.042         0.1300
   2.5     0.028         0.1413
   3.0     0.018         0.1594
   4.0     0.013         0.1803
saved side-by-side for sigma=2.5
```

2.3 — pyramids and reconstruction (`run_2_3_pyramids.txt`)

```
image (670, 736, 3) (sides are not powers of 2)
level  Gaussian shape     pixels   Laplacian shape
  0    (670, 736)         493120   (670, 736)
  1    (335, 368)         123280   (335, 368)
  2    (168, 184)          30912   (168, 184)
  3    (84, 92)             7728   (84, 92)
  4    (42, 46)             1932   (42, 46)
  5    (21, 23)              483   (21, 23)
total pixels in the Gaussian pyramid / image = 657455/493120 = 1.33326  (4/3 = 1.33333); Laplacian pyramid has the same sizes -> same total
Laplacian display: 0.5 + 1.99 * L  (99.5th percentile of |L| over levels 0..4 is 0.2514), clipped to [0, 1]
REBUILD (ours): shape (670, 736, 3); max|error| = 1.665e-16; RMSE = 1.994e-18; exactly zero in 99.6% of the values
       8-bit-quantized input: max|error| = 2.220e-16
REBUILD ('level minus its blur', Lecture-3 style as I understand it): max|error| = 0.2673; RMSE = 0.0492
ALIASING LEFT BY reduce():  one reduce keeps every 2nd pixel, new Nyquist = 0.25 cycles/px; kernel response H(f) = cos^4(pi f)
  sine f=0.30: false pattern amplitude 0.119   (cos^4(pi f) = 0.119)
  sine f=0.35: false pattern amplitude 0.042   (cos^4(pi f) = 0.042)
  sine f=0.40: false pattern amplitude 0.009   (cos^4(pi f) = 0.009)
rings.png reduced twice with our reduce (-> (128, 128)): false-pattern strength = 0.031
   Gaussian sigma=2.0 then every 4th pixel (from 2.2)            : 0.068
   Gaussian sigma=2.25 then every 4th pixel (from 2.2)           : 0.042
   Gaussian sigma=2.5 then every 4th pixel (from 2.2)            : 0.028
   every 4th pixel, no blur                                     : 0.999
```

2.4 — blending: oraple and own blends

```
[oraple] images (512, 512, 3); mask: left half = 1 (white), right half = 0
[oraple] seam width (pixels where image 1 weighs 10%..90%) vs levels: {1: 0, 2: 4, 3: 8, 4: 16, 5: 34, 6: 67, 7: 134, 8: 273}
[oraple] 6 levels; blend range [-0.000, 1.000], 0.00% of values clipped for display/saving
[oraple] level-by-level figure: Laplacian levels shown as 0.5 + 8.14 * x, clipped to [0, 1]
[own1] 5 levels; blend range [-0.003, 1.000], 3.37% of values clipped for display/saving
[own1] level-by-level figure: Laplacian levels shown as 0.5 + 1.90 * x, clipped to [0, 1]
[own2] 3 levels; blend range [-0.042, 1.066], 0.05% of values clipped for display/saving
```

3.1 — seam carving of the tower (`run_3_1_seams.txt`)

```
broadway_tower (484, 714, 3); removing 214 seams = 30.0% of the width -> 500 columns
possible vertical seams: between 10^148 and 10^233;  DP work per seam: H*W*3 = 1,036,728 comparisons
finite-difference energy: 27.8 s for 214 seams (130 ms per seam incl. energy recompute); result (484, 500, 3)
mean energy per seam pixel: first seam 0.0084, last seam 0.0188, image mean energy 0.0530
PADDING OF THE ENERGY:  zero (as in 1.2)  vs  reflect
  zeros   : energy on the 1-px frame = 0.459 (interior mean 0.053); seam pixels in the 5 leftmost / rightmost columns: 1717 of 103576; mean
seam energy 0.0166
  reflect : energy on the 1-px frame = 0.025 (interior mean 0.053); seam pixels in the 5 leftmost / rightmost columns: 2565 of 103576; mean
seam energy 0.0148
```

3.2 — comparisons for the starter images and our own images

```
[rocket] 640x427 -> width 448 (192 seams, 90 s for both energies) | mean seam energy / image mean energy: FD 0.13 (last seam 0.20), DoG 0.22
(last 0.29) | FD/DoG removed-pixel overlap (Jaccard) 0.43
[home] 512x384 -> width 358 (154 seams, 47 s for both energies) | mean seam energy / image mean energy: FD 0.15 (last seam 0.33), DoG 0.21
(last 0.36) | FD/DoG removed-pixel overlap (Jaccard) 0.53
[starry_night] 720x568 -> width 504 (216 seams, 103 s for both energies) | mean seam energy / image mean energy: FD 0.45 (last seam 0.54),
DoG 0.40 (last 0.50) | FD/DoG removed-pixel overlap (Jaccard) 0.36
[broadway_tower] 714x484 -> width 500 (214 seams, 80 s for both energies) | mean seam energy / image mean energy: FD 0.28 (last seam 0.35),
DoG 0.31 (last 0.42) | FD/DoG removed-pixel overlap (Jaccard) 0.56
[own_person] 612x407 -> width 428 (184 seams, 59 s for both energies) | mean seam energy / image mean energy: FD 0.58 (last seam 0.66), DoG
0.55 (last 0.61) | FD/DoG removed-pixel overlap (Jaccard) 0.34
[own_landscape] 626x468 -> width 438 (188 seams, 60 s for both energies) | mean seam energy / image mean energy: FD 0.63 (last seam 0.70),
DoG 0.69 (last 0.76) | FD/DoG removed-pixel overlap (Jaccard) 0.32
[own_fail] 1000x562 -> width 700 (300 seams, 175 s for both energies) | mean seam energy / image mean energy: FD 0.27 (last seam 0.35), DoG
0.30 (last 0.39) | FD/DoG removed-pixel overlap (Jaccard) 0.59
```

3.2 — failure analysis of Starry Night (`run_3_2_failure_analysis.txt`)

```
[starry_night] 720x568 -> width 504 (216 seams, 103 s for both energies) | mean seam energy / image mean energy: FD 0.45 (last seam 0.54),
DoG 0.40 (last 0.50) | FD/DoG removed-pixel overlap (Jaccard) 0.36
cypress proxy (gray < 0.22, columns < 330): 59913 px = 14.7% of the image
  finite differences  : removed 54.0% of the cypress pixels (a uniform 30% resize would remove 30.0%); 26.3% of ALL removed pixels lie on
the cypress
      row 350: cypress width 75 px -> 21 px  (x0.28)
      row 420: cypress width 171 px -> 96 px  (x0.56)
      row 500: cypress width 197 px -> 99 px  (x0.50)
  DoG (sigma=2)       : removed 64.4% of the cypress pixels (a uniform 30% resize would remove 30.0%); 31.4% of ALL removed pixels lie on
the cypress
      row 350: cypress width 75 px -> 1 px  (x0.01)
      row 420: cypress width 171 px -> 76 px  (x0.44)
      row 500: cypress width 197 px -> 77 px  (x0.39)
saved fig_3_2_failure_starry_night.png
```

Bonus — object removal and enlargement (own photo)

```
[own] 612x459; mask 39886 px, widest row 125 px -> k = 125 seams removed (14.4 s), reduced to width 487; grown back by 125 seams (10.7 s) ->
width 612
[own] how often a pixel of the reduced photo is written: k seams from a copy -> at most 2x (57375 pixels written twice, 0 more than twice);
cheapest seam inserted 125 times -> up to 126x (1658 duplicated, 1556 more than twice)
[own] distinct columns of the reduced photo that get stretched (mean over rows): k-from-copy 125.0, repeated-cheapest 3.6  (of k = 125)
```